What Evidence Should a Recommender Change Require? A Controlled Study of Session-Context Gating

Ronan Francis
Atlantic Technological University, Galway
ronananthonyfrancis@gmail.com
Supervisor: Michael Duignan

***Abstract*—What evidence should a claimed improvement to a recommender require? This paper asks it of one change to a sequential recommender: adding session context, such as how often the content category has changed, through a gate alone. CAFREC balances a SASRec encoder against a precomputed long-term profile through a gate that sees only this context. Without the profile it is SASRec plus the gate, which isolates the gate's effect. On KuaiRand-Pure, a short-video dataset, with every item ranked, seven seeds, and paired per-user tests, the gate does not significantly improve SASRec. HR@10, the share of users whose next click ranks in the top ten, changes by $+0.7\%$ (95% interval -0.7% to $+2.2\%$), bounding any undetected benefit at about 2%. The null holds after equal-budget tuning, at configurations of different depth. An earlier setup with SASRec at batch size 2048 and the gated model at 512 showed an apparent $+3.7\%$ advantage, positive in all seven seeds ($p = 4.5 \times 10^{-4}$). Only matching the settings exposed it. For the least active users, the planned hypothesis is inconclusive: a language-model-generated profile trailed a profile-free model by 12.6%. Its prompt describes the items the model is trained to predict, an unmeasured training-target leak expected to work against the profile. Here the seed, batch size, and loss each moved results more than the gate did. For small offline effects, in publications and Digital Services Act risk assessments, the paper proposes a minimum standard: matched settings recorded for every run, several seeds per configuration, and paired tests with intervals.**

***Index Terms*—Sequential recommendation, session context, gating, user modelling, ablation study, offline evaluation, reproducibility, negative results, short-video recommendation.**

I. Introduction

SHORT-VIDEO platforms log interactions at a granularity that exposes rapid shifts of intent within a session. Sequential recommenders condition on the order of recent interactions [1], [2], and hierarchical variants separate within-session dynamics from cross-session preferences [3]. Combining long- and short-term signals raises the stability–plasticity dilemma [4]: a model must keep a stable picture of lasting taste yet adapt quickly to a change of intent. Dual-branch architectures, in which a learned gate balances a long-term branch against a short-term one, are a common response [5], [6]. In most such designs the gate is conditioned on the representations being fused. An alternative is to condition it only on exogenous properties of the session, measured from the log rather than computed by the model. Examples are how many interactions the session holds so far, how varied the user's attention has been, and how often the content category has changed.

Context has been admitted into sequential recommenders through self-attention or the item representation [7], [8], into the weight that fuses long- and short-term representations [9], and into gated and attention-based session recommenders [10]–[12]. In those models context enters inside the encoder or alongside the representations from which the fusion weights are computed. The session models' comparisons with context-free variants are single-run point estimates, without seed variance or significance tests (Section II-C). Industrial recommenders also gate on exogenous features, conditioning embeddings, hidden units, or whole modules on personalised prior or user-group features [13], [14]. Outside recommendation, feature-wise modulation of an intermediate representation by a conditioning input is a general mechanism [15]. Each of these changes the network throughout. None of the studies above holds a sequential encoder fixed with the unmodified encoder as the limiting case of the fusion. This paper asks what evidence a claimed change to a recommender should require, and answers it through one controlled question:

> *Does exogenous session context provide a detectable benefit when it is introduced only through a context gate, while the underlying sequential recommender is held fixed?*

On KuaiRand-Pure the answer is: not detectably, at either of the two operating points evaluated. The data bound any benefit that could have escaped detection at about 2% of SASRec's HR@10, the share of users whose held-out next click is ranked in the top ten. An operating point is a choice of hyperparameters for both models. The first uses shared defaults. The second comes from an equal-budget search, which gives both models the same configurations to choose from, and at it the selected configurations differ in depth (four layers for SASRec, two for the gated model). A secondary question is whether the evaluated long-term profile representations add value beyond the short-term sequential encoder. No long-term profile gained over the model without one. The seed, the batch size (Section IV-F), and the loss each moved results by

R. Francis is with the Department of Computer Science and Applied Physics, Atlantic Technological University (ATU), Galway, Ireland (e-mail: ronananthonyfrancis@gmail.com).

This paper is submitted in partial fulfilment of the requirements for the M.Sc. in Computing.

more than the gate did. From this, Section VII-A proposes a minimum evidence standard for small effects measured offline.

This paper uses CAFREC, the Context-Adaptive Fusion Recommender, as an instrument for a controlled comparison rather than proposing a generally superior recommender. CAFREC combines a SASRec short-term encoder with a long-term user profile through an element-wise gate whose *only* input is a session-context vector. The profile is frozen, that is, computed offline and not updated during training. At the default operating point CAFREC's encoder, loss, batch size, and training schedule are identical to SASRec's. The gate's output g balances the profile against the encoder in each dimension, and SASRec is the limiting case of the fusion as $g \to 0$, when only the encoder's output remains. Removing the profile gives context-gated SASRec (CAFREC-NP), which differs from SASRec only by the gate. With nothing else changed, the long-term branch can be removed, replaced by a trainable embedding of equal size, or re-encoded with a larger model. It can also be replaced by a semantics-free control or by profiles of watched content. Every comparison uses full-catalogue ranking on KuaiRand [16], in which each user's held-out item is ranked against every item rather than a sample. Significance tests are paired per user: each user's result under one model is compared with the same user's result under the other. The headline comparison is repeated over seven training seeds.

The study was organised around four hypotheses. The first three were stated in the project plan before any experiment was run, and the fourth was added in a later revision of the plan (Section IV-H).

- **H1 (architectural separation):** dual-network architectures with explicit long- and short-term components will outperform the sequential baselines (HGN [17], SASRec [1], HGRU4Rec [3]) on HR@10 and NDCG@10, including on the short-session cohort. *Pre-specified; not supported against SASRec (Section VI-A).*
- **H2 (semantic enrichment for sparse users):** LLM-generated temporal user profiles will achieve higher NDCG@10 than interaction-derived embeddings for users in the lowest quartile of interaction counts. *Pre-specified; inconclusive (Section VI-C3).*
- **H3 (context-adaptive versus static fusion):** context-adaptive gating will outperform both static-weighted and concatenation-based fusion across all activity strata, with the largest margin on high-category-drift sessions. *Pre-specified; partly supported, mechanism not established (Section VI-D).*
- **H4 (diversity under adaptive gating):** context-adaptive gating will yield measurably higher content diversity, as measured by intra-list diversity (ILD) and coverage, compared with static fusion baselines, with the largest gains observed in high-category-drift sessions. *Added in the August revision, after the first development runs; partly supported (Section VI-F).*

H1's third baseline is realised here as a user-seeded GRU over the recent-item window, after HGRU4Rec [3], and is named U-GRU below (Section IV-D). H4 is a directional prediction rather than a mechanistic one: the plan specified no mechanism by which context gating would raise diversity.

Coverage and intra-list diversity also carry a governance motivation. Article 34 of the Digital Services Act [18] requires very large online platforms to assess systemic risks stemming from their services, taking into account the design of their recommender systems. Article 35 requires them to put in place mitigation measures, which may include testing and adapting those systems. Coverage and intra-list diversity are offline quantities such an assessment can use. A platform would need to establish whether a gating mechanism moves them, and by how much relative to seed variation. Section VII-A states the evidence standard this implies.

The contributions are empirical and methodological:

- **Measured sources of spurious small effects, and a proposed minimum evidence standard.** Multiple runs per configuration, reported seed variance, and significance testing are established recommendations in machine learning and language processing [19]–[21]. Tuning effort and baseline configuration are known sources of apparent progress in recommendation [22]–[24]. On one matched recommender comparison, this study measures how far each of the settings above, and the device, moves a result. It shows that matched settings, several seeds, and paired tests each catch a failure the other two miss. It extends to HGN and a user-seeded GRU the finding that the loss can reverse comparisons between sequential architectures [25]. It also applies the three practices to claims of small gains in publications and in Digital Services Act assessments (Section VII-A).
- **Controlled isolation of a context gate, with a bounded null.** In this design SASRec is the $g \to 0$ limit of the fusion, so context-gated SASRec differs from its baseline only by an exogenous gate. Encoder, loss, batch size, and schedule are matched at the default operating point. Seven seeds and 22,912 paired users bound the gate's effect on full-catalogue accuracy (Section VI-A). Fusion ablations within activity and drift strata and two gate controls test what the gate depends on. Coverage and intra-list diversity measure its effect beyond accuracy. Context-conditioned gating itself is not new; the contribution is this isolation.
- **A constant-profile control, with profile ablations.** In the lowest activity quartile, a frozen vector identical for every user reproduces the whole advantage a frozen profile holds over a trainable per-user embedding. That advantage is therefore not personalisation. The finding is exploratory and rests on three seeds. What transfers is the control: a constant vector as the data-minimal baseline any per-user profile study can report. The other ablations remove the long-term branch, replace it with a trainable embedding of equal size, and re-encode it with a 7B-parameter encoder. They also substitute a semantics-free hashing control, three content-bearing profiles, and a generated profile. None gains over the model without a long-term branch in a primary test.
- **A released harness with analysis-status reporting.** Each analysis is classified as pre-specified, added after

the first results, or exploratory, and deviations and unexecuted analyses are listed. The harness, with its test suite and causal session-context features for KuaiRand (Section V), is released at https://github.com/Ronan-Francis/CAFREC.

Section II reviews related work, and Section III describes CAFREC and the variants derived from it. Section IV sets out the data, protocol, statistical tests, and status of each analysis, and Section V describes the harness. Section VI reports the results, Section VII states what they establish and their limits, and Section VIII concludes.

II. Related Work

The work discussed here was identified by a structured search for hybrid user-modelling methods in video recommendation published between 2021 and 2026. It covered IEEE Xplore, the ACM Digital Library, Scopus, and Google Scholar. After deduplication by title and DOI the search returned 342 records. Screening excluded 255 of them as off-topic and 64 of the remaining 87 for low rigour, which left 23 peer-reviewed studies. The search protocol and screening decisions are documented separately [26].

A. Long- and Short-Term Modelling and Gated Fusion

Continual-learning methods address the stability–plasticity dilemma through regularisation, architectural isolation, or replay. Dual-network designs split responsibility between a stable component that preserves earlier knowledge and a plastic one that acquires new knowledge [5]. In recommendation, early deep models of interaction order did not separate long- and short-term components. These were first recurrent networks over session data [27] and then self-attentive models [1], [2]. Hierarchical recurrent models introduced an explicit cross-session user state [3], and later work added cascaded cross-attention [28]. Gated fusion recurs across this line of work as the mechanism for adapting to within-session behavioural change. SLi-Rec fuses long- and short-term representations with a weight computed from both representations and from contextual features such as the time interval and the target item [9]. Its weight therefore does not depend on context alone. Rec-SSP computes its gate from the long- and short-term representations and reports lower rating-prediction error than concatenation, multiplication, or averaging [6]. In these designs the gate sees the representations being fused.

B. LLM-Based User Profiles

LLM-based temporal profiling generates natural-language summaries of user behaviour. These can capture how a user's topics change over time, which interaction vectors do not encode [29]. H2 expected the advantage of such profiles to be largest for sparse users. Sabouri et al. in fact found it smaller on the sparser of their two datasets [29]. Generating the profiles also carries a non-trivial inference cost. Four of the five profiles evaluated here are simpler relatives of this line of work rather than instances of it. The pre-specified profile embeds a templated summary of behavioural statistics with a pretrained text encoder. The content-bearing profiles average category or caption representations (Section III-C). The fifth is generated by a language model and supplies the condition H2 names. There the advantage expected for sparse users did not appear, in a test that Section VI-C3 reports as inconclusive.

C. Context-Aware Gating in Session Recommendation

Sequential recommenders admit context through the encoder. TiSASRec adds the time intervals between interactions to self-attention [7]. The only context it admits is therefore time, and it admits it by changing the encoder. CARCA embeds each interaction's contextual features, such as its timestamp, together with the item before self-attention. It then scores candidates by cross-attention [8]. Session-based recommenders have also incorporated contextual information into gated architectures. CA-GGNN replaces the input and state matrices of the GRU inside a gated graph neural network with two context matrices [10]. One holds context observed in the session, such as time and location, and the other is an interval context giving each item's share of the session's browsing time. CA-GGNN also adds the context to the GRU's update gate. Later context-aware session models admit context through attention and graph structure instead. C-HAN weights each item's interaction contexts, such as weekday, month, and a working-day indicator, by attention [11]. It concatenates the weighted context embedding with the item embedding before a hypergraph attention network. That branch is then fused with a self-attention branch through a soft-attention weight computed from the two representations. CARES builds a cross-session graph whose edge types are defined by item categories, and aggregates it with relation-aware graph attention [12]. It personalises each item's representation through a gate computed from the item and session embeddings.

In all three session models the weights that combine branches are computed from the representations being combined, and context is one more input to them. Each study compares the full model with a variant without context: CA-GGNN with its context-free base SR-GNN, C-HAN with C-HAN-C, and CARES with a variant without category information. Each reports single-run point estimates without seed variance or significance tests. CARES also takes its baseline results from the original papers [10]–[12].

Feature-conditioned gating is also used in industrial recommenders. PEPNet gates embeddings and hidden units on personalised prior features and is deployed at Kuaishou, the platform behind KuaiRand [13]. POSO gates modules on user-group features such as new versus regular users [14]. The underlying operation also appears outside recommendation. In visual reasoning, FiLM scales and shifts each feature by amounts computed from a conditioning input [15]. In each case the conditioning enters at many points in the network rather than at a single fusion step.

D. Diversity and Evaluation Practice

Two strands of work bear on H4 and on the evidence standard of Section VII-A: methods that broaden what a recommender shows, and the reliability of offline comparisons

between models. Multi-objective reinforcement learning improves coverage without proportional accuracy loss [30], but has not been evaluated within gated-fusion architectures. Offline evaluation practice adds its own uncertainty. Once tuned, simple nearest-neighbour and graph-based baselines match or outperform most of the recent neural methods re-examined by Ferrari Dacrema et al. [22]. Metrics that rank the held-out item against a sample of negatives are not guaranteed to preserve the ordering produced by full-catalogue ranking [31]. Such protocol choices are a documented source of inconsistency across published results [32]. Training choices matter as much: SASRec trained with full cross-entropy outperforms BERT4Rec, reversing the comparison reported with SASRec's original loss [25]. Implementations of BERT4Rec, RecBole's among them, fall well short of the originally reported results at their default configurations. The original code needs far more training than its default to reach them [33].

The 23 studies retained in the literature review [26] were coded from the papers themselves on five points of evaluation practice. They do not include the three session models of Section II-C. Seven are surveys, and four could not be coded: two could not be obtained in full text and two could not be located under the titles cited. Of the remaining 12 empirical studies, six report results averaged over repeated runs or folds, one of them with a fixed random seed. Two report a measure of variance, one only for its analysis tables. Two name a significance test, and three state a p-value threshold without naming the test. Seven report no test, and three of those nevertheless describe their gains as significant. Of the seven that rank items, three rank the held-out item against 99 or 100 sampled negatives, two rank the full catalogue, and two do not say which. The other five predict ratings or classify images. All 12 ran their own baselines rather than copying published results, except that one study took a single baseline from its original paper. No study combines repeated runs with a significance test, two of the three practices Section VII-A asks for. The three session models of Section II-C, outside this count, each report single runs without a significance test. None of these 15 empirical studies therefore combines the two practices.

Evaluation practice also matters outside research. Under Article 37 of the Digital Services Act, providers of very large online platforms and search engines are audited independently at least once a year [18]. Commission Delegated Regulation (EU) 2024/436 lays down how those audits are performed [34]. Its recitals ask that audit methodologies for algorithmic systems include tests, for example tests that independently reproduce and assess the results of accuracy indicators (Recital 30). For sampling from a personalised system, they give two examples of adequate practice: sampling from the cohorts that personalisation creates, or identifying the margin of error and justifying why it is acceptable (Recital 33). Article 12 requires the audit report to justify the sample size and sampling method. The algorithmic-audit literature describes two forms such work can take. Internal audits are embedded in the development lifecycle, and each stage yields documents that together form the audit report [35]. External audits infer a system's behaviour by querying it systematically and observing its outputs [36]. The Regulation's provisions concern the sampling of data and the design of tests, and do not mention the variation that retraining a model introduces. In an offline comparison between two configurations of a recommender, that variation can exceed the effect under test. Section VII-A proposes a minimum for that case as a methodological standard, not as a reading of what the Regulation requires.

E. Positioning of This Study

Table I compares CAFREC with the seven models closest to it. In each of them the fusion weight, where one exists, is conditioned on sequential patterns or on the representations being fused. None reports an explicit mechanism to preserve content diversity. CAFREC's gate is an ordinary multilayer perceptron, and conditioning a gate on context is not in itself new (Section II-C). The contribution of this paper is therefore empirical and methodological rather than algorithmic. It isolates exogenous session-context gating from any change to the underlying sequential encoder or loss. SASRec stays inside the gated model's function class, as the $g \to 0$ limit of the fusion. The study evaluates that gating with matched full-catalogue ranking, multiple seeds, paired user-level tests, and explicit ablations of the long-term profile. The context-aware models above do not provide that isolation. In them context changes the encoder, its input, or the inputs to the fusion weight, and the session models' comparisons are single runs. The industrial models condition on exogenous features, as CAFREC does, but change the network throughout. None of them holds a sequential encoder fixed with the unmodified encoder as the limiting case of the fusion. The claim concerns where the conditioning enters, not whether training drives the gate to that limit.

III. The CAFREC Model

A. Overview

CAFREC is a sequential recommender composed of three branches (Fig. 1). A short-term encoder f_{short} summarises the user's recent interaction window into $z_{\text{short}} \in \mathbb{R}^H$. A long-term branch projects a precomputed, frozen user profile into the same space as $z_{\text{long}} \in \mathbb{R}^H$. A gating network f_{gate} maps a vector of session-context features x_{ctx} to a gate $g \in [0,1]^H$ that mixes the two. The fused representation

$$z \;=\; g \odot z_{\text{long}} \;+\; (1-g) \odot z_{\text{short}}, \qquad g = f_{\text{gate}}(x_{\text{ctx}}), \quad (1)$$

is scored against the item embedding matrix to produce a full-catalogue ranking.

At the default operating point, the short-term branch is *identical* to the SASRec baseline [1], down to the number of layers, attention heads, inner dimension, dropout rates, and initialisation. It is trained with the same loss, batch size, and schedule (Section IV-F). A difference between CAFREC and SASRec at that operating point can therefore come only from the long-term branch and the gate, not from a more powerful or better-tuned sequence model.

Two limiting cases of (1) organise the study. SASRec lies in CAFREC's function class as the limit of the fusion when $g \to \mathbf{0}$. Whether training approaches that limit is a separate

TABLE I
COMPARISON OF CAFREC WITH THE SEVEN MODELS CLOSEST TO IT

Model	Mechanism	Input	Where context enters	Fusion conditioned on	Key limitation
TiSASRec [7]	Self-attention	Sequences, time intervals	Self-attention	–	Context limited to time; enters the encoder
SLi-Rec [9]	Gated fusion	Both branches, context	Fusion weight	Both representations, context, target	Fusion weight not conditioned on context alone
Rec-SSP [6]	Gated fusion	Multi-level	No context	Fused representations	High cost of sequence-pattern analysis
CA-GGNN [10]	Gated graph network	Sessions, context matrices	GRU input, state, update gate	–	Single-run comparison
CARES [12]	Relation-aware graph attn.	Cross-session graph, categories	Item representation, by a gate	Item and session embeddings	Single run; baselines from original papers
PEPNet [13]	Feature-wise gating	Features, prior features	Embeddings and hidden units	–	Conditioning throughout the network
C-HAN [11]	Hypergraph attention	Sessions, interaction contexts	Item representation, by attention	Both representations	Single-run comparison
CAFREC (this study)	Gated fusion	Sequences, session context	Fusion gate only	Session context only	No aggregate gain; one tier, one leave-one-out split, ten epochs

A dash marks a column that does not apply to that model.

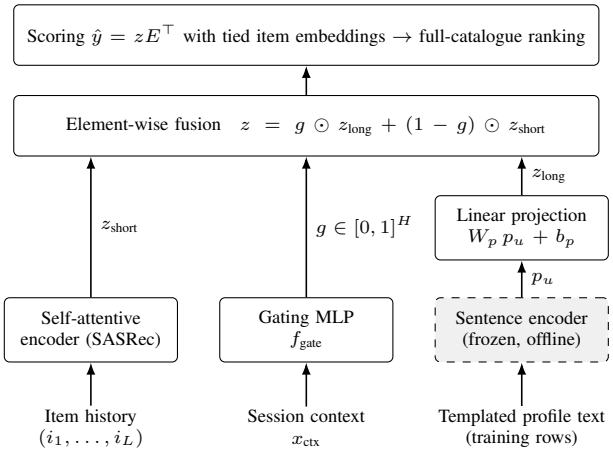


Fig. 1. CAFREC architecture with the pre-specified profile. The short-term self-attentive encoder and the frozen text-embedded profile are combined by an element-wise gate whose sole input is the session-context vector x_{ctx}. Setting $z_{\text{long}} = \mathbf{0}$ gives CAFREC-NP; $g \to \mathbf{0}$ recovers SASRec in the limit. The item embedding matrix E is shared between the encoder input and the scoring head. Dashed: frozen component.

question, and the gate is initialised near $g = 0.5$ (Section III-D). With the long-term branch removed ($z_{\text{long}} = \mathbf{0}$), (1) becomes

$$z = (1-g) \odot z_{\text{short}}, \qquad g = f_{\text{gate}}(x_{\text{ctx}}), \tag{2}$$

a context-conditioned, per-dimension modulation of the SASRec state. In FiLM terms [15], it scales z_{short} without shifting it, with the scale constrained to $[0, 1]$ and a conditioning input exogenous to both branches. This variant is termed **CAFREC-NP**, or *context-gated SASRec*. Comparing CAFREC-NP with SASRec isolates the effect of session-context gating, and comparing CAFREC with CAFREC-NP isolates the effect of the long-term profile.

B. Short-Term Sequential Encoder

The short-term branch is a self-attentive encoder over the causal item history [1]. Each user's item sequence $s = (i_1, \ldots, i_L)$ is truncated to the $L \leq 20$ most recent interactions. The input representation sums a learned item embedding and a learned absolute position embedding,

$$E_t^{(0)} = \text{Dropout}\big(\text{LayerNorm}(\mathbf{e}_{i_t} + \mathbf{p}_t)\big), \quad t = 1, \ldots, L \tag{3}$$

which is passed through N Transformer blocks under a lower-triangular attention mask. The mask prevents any future interaction from affecting the representation of the current one. The short-term state is the output at the final valid position, $z_{\text{short}} = E_L^{(N)} \in \mathbb{R}^H$. The model uses $H = 64$, $N = 2$ layers, two attention heads, feed-forward inner size 256, hidden and attention dropout 0.5, GELU activation, and initialisation from $\mathcal{N}(0, 0.02^2)$. The history window is capped at 20 interactions for compute reasons and applied to all models. The one comparison that varies it ($L \in \{10, 20, 50\}$, three seeds) trained SASRec at batch size 2048 and the CAFREC variants at 512. That confounds window length with batch size, so its results are not reported (Table III).

C. Long-Term Branch

a) Pre-specified profile: Each user is represented by a single vector computed offline from a templated text summary of that user's training history. The idea loosely follows the temporal profiling of Sabouri et al. [29]. The text is filled in by a fixed template and is *not* generated by a language model. It states a history-length category (sparse, moderate, or rich, with boundaries at 10 and 40 interactions) with the raw interaction count. It gives the user's mean session depth, dwell-time entropy, category drift, and inter-session return gap, each bucketed as low, moderate, or high at ± 0.75 standard deviations. It also gives the share of context rows that follow a policy transition and the share of first-session activity. A frozen pretrained text encoder embeds the string to give $p_u \in \mathbb{R}^{d_p}$.

The profile is a user-level aggregate of the behavioural statistics the gate observes per session, and contains no item identities. It is a deliberately limited representation of long-term preference. It was retained because it was the long-term branch specified in the project plan and was fixed before its

results were known. The content-bearing profiles below record *what* a user watched instead.

The construction consumes only training rows (every interaction except each user's last two, held out for validation and testing; Section IV-E). It is therefore causal with respect to the validation and test items: no profile encodes an item the model is scored on. It is *not* causal with respect to training examples, as the paragraph on the training-target leak below describes. The profile is also *row-aligned*. Profiles are stored in the framework's internal user order as one $n_{\text{users}} \times d_p$ tensor. A shape and dtype check on load rejects any tensor that does not match the dataset.

b) Text encoders: The template text is embedded with `bge-large-en-v1.5` (335M parameters, $d_p = 1024$) [37], a BERT-style sentence encoder. To test whether encoder capacity limits the profile, it is also embedded with `e5-mistral-7b-instruct` (7B parameters, $d_p = 4096$, fp16) [38], an embedding model fine-tuned from a decoder-only LLM. Neither generates text. Both cover all 22,912 KuaiRand-Pure users retained after filtering (Section IV-B). The profile tensor carries one further row, because the framework's internal user index reserves index 0 for padding. The shape check is made against that count. A deterministic CRC32 bag-of-tokens embedding of the same text at $d_p = 1024$ serves as a no-semantics control.

c) Content-bearing profiles: Three further profiles were added after the result for the pre-specified profile was known, to test the dense cohort (Section VI-C1). Each describes the videos a user watched in training rows. Two use only *beyond-window* rows: besides the validation and test rows, they drop the 20 rows immediately before them, which the short-term encoder attends over at test time. Users with at most 22 rows receive a zero vector. `cat_beyond` ($d_p = 38$) is the L_1-normalised histogram of first-level video categories over those rows. `cap_beyond` ($d_p = 1024$) is the L_2-normalised mean of caption embeddings over them, each caption embedded once with `bge-large-zh-v1.5` [37]. `cap_all` is the same mean over all training rows and gives every user, including the least active, a non-zero vector. The beyond-window profiles are non-zero for 88.5% of dense users and for no sparse user. None of the three is generated by a language model.

d) LLM-generated profile: A fifth profile tests H2 as stated in the plan. An open-weight instruction-tuned language model, `Qwen2.5-7B-Instruct` [39], writes one English paragraph per user. The prompt holds that user's bucketed behavioural statistics, which are the fields the templated profile encodes. It also holds the category mix over the user's training rows, and the first-level category and Chinese caption of the 32 most recent of them. Decoding is greedy at temperature 0 under a 200-token cap, which makes the corpus a deterministic function of the prompts and the model revision. Of the 22,912 generations, 84 reached that cap, and the paragraphs average 97 words. The prompt asks for category names in English so that an English encoder is not asked to represent Chinese tokens. The paragraphs are embedded with `bge-large-en-v1.5`, the encoder used for the templated profile, so a comparison between the two isolates the text rather than the encoder. Construction uses every training row, as the pre-specified profile does, rather than the beyond-window rows of two content-bearing profiles. That window excludes every user with 22 rows or fewer, so it would leave the entire lowest activity quartile with a zero vector. Under that condition `cap_beyond` was uninformative. Every user receives a profile.

e) Training-target leak: The framework enables sequential data augmentation, which makes every prefix of a training sequence itself a training example. A profile built from training rows that follow a prefix encodes that example's target, although no profile encodes a validation or test item. This paper calls this the *training-target leak*, and profiles are exposed to it to different degrees. The generated profile is the most exposed. Its prompt names the category and caption of the 32 most recent training rows. For a user with at most 34 organic clicks these are every training row, which covers the whole lowest and second activity quartiles. For these users the prompt describes the target of every augmented training example. `cap_all` averages every target into one vector. `cat_beyond` and `cap_beyond` average in only the targets that lie outside the last 20 training rows. The templated, 7B-encoded, and hashing profiles, and the history-length input of CAFREC-H (Section VI-G), carry the leak only through behavioural aggregates or a count, with no item identity. That is the weakest exposure. The leak rewards the long-term branch during training for a signal it lacks at test time, so it is expected to work against the generated profile. No experiment here measures its size. Nor can the H2 test separate a profile that carries no useful signal from one fitted to that signal (Section VI-C3).

f) Projection and freezing: The profile is projected into the hidden space by a learned linear map, $z_{\text{long}} = W_p\, p_u + b_p$ with $W_p \in \mathbb{R}^{H \times d_p}$. The profile table is frozen and only W_p and b_p are trained, which makes z_{long} an *external* representation. A trainable table would instead be a per-user free parameter fitted to the recommendation objective. That is the trainable stand-in control of Section VI-C.

D. Context-Adaptive Gating

The gate input $x_{\text{ctx}} \in \mathbb{R}^6$ has six features (Table II). Four are core session-context features: session length, dwell-time entropy, category drift rate, and a policy-transition flag. Two are auxiliary session-recency features: the inter-session gap and a first-session indicator. Every feature is computed on the session *prefix* and reflects only events that have already occurred. The gate is a three-layer perceptron,

$$g = \sigma\big(W_3\, \phi(W_2\, \phi(W_1 x_{\text{ctx}} + b_1) + b_2) + b_3\big), \qquad (4)$$

with ReLU ϕ, hidden width 64, and output width $H = 64$. The gate is initialised with the rest of the model: linear weights from $\mathcal{N}(0, 0.02^2)$ and biases at zero. That puts the pre-sigmoid logit near zero and $g \approx 0.5$ at the start of training. CAFREC-NP therefore begins at $0.5\, z_{\text{short}}$, which halves every score relative to an identically initialised SASRec. That is equivalent to a softmax temperature of 2 (Section VII-B). The SASRec limit $g \to \mathbf{0}$ is reached only asymptotically, if training moves towards it at all.

The gate's input is *exclusively* x_{ctx}: it does not see z_{short}, z_{long}, the item sequence, or the user identity. Gated-fusion designs that condition on the representations being fused [6] can learn which branch looks more confident. Conditioning on exogenous context is intended to make the gate respond to what kind of session is in progress. Whether the trained gate actually uses session-specific information is an empirical question, tested by the controls in Section VI-D. As a vector rather than a scalar, the gate can route different latent dimensions to different branches within one session. A scalar gate is retained as an ablation.

E. Fusion and Scoring

Fusion follows (1). In it, $g \to 1$ favours the long-term profile and $g \to 0$ the short-term encoder, which makes g a per-session stability–plasticity setting by design [4]. Scoring reuses the item embedding matrix E as the output head, $\hat{y} = zE^\top \in \mathbb{R}^{|I|}$. Training minimises full-softmax cross-entropy over the catalogue without negative sampling,

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(s,i^+)\in B} \log \frac{\exp(z^\top \mathbf{e}_{i^+})}{\sum_{j\in I} \exp(z^\top \mathbf{e}_j)}. \tag{5}$$

This is SASRec's training contract, and all models share one batch size.

IV. Research Methodology

A. Research Design

The study is a controlled ablation. The treatment, a gate conditioned only on session context, is added to a fixed SASRec encoder. The baseline is the treatment's own limit as $g \to \mathbf{0}$ (Section III-A). At the default operating point the two arms therefore share encoder, loss, batch size, schedule, data, and evaluation, and differ only by the gate. Analyses are separated by when they were specified (Table III). Pre-specified analyses and those added after the first results are reported as tests, with Holm adjustment within each hypothesis's family. Exploratory analyses are reported as hypothesis-generating, with no conclusion resting on them. Inference is paired over users and repeated over seeds. The variation between training runs of one neural model can be as large as the differences reported between methods [19], [21], so one run per arm may not tell the two apart. So it proved here: the seed-to-seed standard deviation of HR@10 is more than twice the effect under test (Section VI-A). Evaluation is offline, on a public log with full-catalogue ranking, because no online A/B test on a live platform was available to this study. The findings therefore concern offline accuracy and diversity, not the behaviour of users exposed to the model.

B. Dataset

KuaiRand [16] is an interaction log from the Kuaishou short-video platform. Its per-impression `is_rand` indicator marks each impression as a standard delivery or a random-policy intervention, which allows organic behaviour to be isolated. Logs without such an indicator, such as the unified search-and-recommendation log KuaiSAR [40], offer no way to separate organic impressions from any injected exposures.

Results are reported on KuaiRand-Pure (27,285 users and 7,551 items as released), whose catalogue is small enough for full-catalogue ranking of every model. Training and evaluation targets are organic clicks, that is, standard-policy impressions the user clicked. Users with fewer than five organic clicks are removed, leaving 651,099 rows over 22,912 users and 7,210 items. Sessions are defined by a 30-minute inactivity threshold on `time_ms`, fixed before any feature computation and not tuned. The larger KuaiRand-1K and 27K tiers need a k-core item floor before they yield a trainable catalogue and are left to future work.

The dataset is released under CC BY-SA 4.0 with user and video identifiers re-indexed and some user features encrypted [16]. The study involves no human participants and no primary data collection, and no re-identification is attempted. Fairness across protected groups cannot be assessed, because KuaiRand contains no labelled demographic attributes.

a) Ethics: The generated profiles of Section III-C carry risks this study did not evaluate. The generator's prompt contains third-party caption text, and a caption can carry instructions to the generator, an indirect prompt injection [41]. The profile cache embeds paragraphs that name each user's watched categories and captions. Inverting the embeddings of another text encoder, GTR-base, has recovered 92% of 32-token inputs exactly [42]. Inversion of `bge-large` embeddings was not tested. Where such data are personal data, Article 17 of the GDPR [43] gives a right to erasure on the grounds it lists. A request on one of those grounds covers the generated paragraphs and the profile cache as well as the source rows, unless the research exemption of Article 17(3)(d) applies. The Article does not state whether the right covers trained model parameters.

b) Cohorts and partitions: Users with fewer than 20 organic clicks form the sparse cohort (10,345 test users, 45.2%) and the remainder the dense cohort (12,567). The median user has 22 organic clicks, and activity is right-skewed. The hypotheses require three further partitions of the test users. They were fixed in the analysis code after the aggregate results were known but before any stratified result was inspected. *Activity quartiles* split users by organic clicks at 12, 22, and 38, and the lowest quartile holds 6,240 users. *Category drift* splits held-out interactions by their session prefix. The 3,521 whose prefix contains at least one category change (raw drift rate above 0.1) are high drift, and the remaining 19,391 are low drift. *Session position* separates the 15,706 users whose held-out interaction opens a session from the 7,206 whose held-out interaction follows earlier interactions in the same session. H1 names a short-session cohort, but this partition is defined by session *position*, not length: an interaction that opens a session need not belong to a short one. The first group is called *session openers* below. H1's wording is used only where it is quoted. Openers are 68.5% of held-out interactions (15,706 of 22,912), and for all of them the within-session features take their empty-prefix values. Sessions are cut on the full impression log rather than on clicks alone (Section IV-C). A held-out click opens a session when no impression of any

kind preceded it within 30 minutes.

Three activity thresholds appear in this study, and they are not interchangeable. The profile template's history-length buckets are at 10 and 40 interactions (Section III-C), the sparse/dense cohort boundary is at 20 organic clicks, and the lowest activity quartile is at 12 or fewer.

C. Context Features

Table II maps each component of x_{ctx} to its KuaiRand source. Continuous features are z-scored with means and standard deviations fitted on training rows only ($605{,}275 = 651{,}099 - 2 \times 22{,}912$ rows). The persisted scaler is applied unchanged to validation and test rows. Those training rows include other users' interactions that post-date some test clicks, the timeline leak of Section IV-E. The scaler, however, passes on only four global means and four standard deviations. Context is computed over the full impression log (2,622,668 rows, including random-policy impressions), whereas targets are organic clicks only. Over organic rows alone the policy flag would be identically zero, and over impressions 0.98% of target rows carry it.

The drift feature reads KuaiRand's `tag` field, whereas ILD and the content-bearing profiles read the first-level categories of the supplementary category file (Section IV-E). The supplementary file was added after the context features had been computed and persisted, and they were not recomputed. The `tag` field is retained as a sensitivity check for ILD.

KuaiRand has no search interface. The policy flag, a within-session `is_rand` boundary, therefore serves as an indirect surrogate for the search-to-recommendation transitions of unified logs such as KuaiSAR. Both mark content arriving from outside the user's organic trajectory, but a search crossing is user-initiated and a policy transition is platform-initiated. The flag is one of four core features, and its planned ablation is reported in Section VI-B.

D. Baselines

The primary baseline is SASRec [1], whose encoder, loss, and training schedule are identical to CAFREC's short-term branch at the default operating point. Every claim about the effect of context gating rests on this comparison. Two further baselines separate long- and short-term signal. HGN [17] combines feature- and instance-level gating with a user embedding. U-GRU, a user-seeded GRU after HGRU4Rec [3], propagates a cross-session recurrent user state of its own. Each is trained twice. The pre-specified run uses its native pairwise BPR loss [44] with one sampled negative. A second run uses SASRec's full-softmax cross-entropy, so that architecture can also be compared at a matched loss.

E. Evaluation Protocol

Interactions are ordered by timestamp per user. In this leave-one-out split, the final item is held out for testing and the second-last for validation. Because each user is split at their own last click, other users' training rows can post-date a test click. Section VII-D quantifies this *timeline leak* [45] and gives an example. At test time the held-out positive is ranked against every item in the catalogue, with previously interacted items masked. Full ranking is used because sampled metrics are not guaranteed to preserve the ordering it produces [31], [32]. Writing r_u for the rank of user u's held-out item,

$$\text{HR@10} = \frac{1}{|U|} \sum_{u \in U} \mathbf{1}[r_u \le 10], \tag{6}$$

$$\text{NDCG@10} = \frac{1}{|U|} \sum_{u \in U} \frac{\mathbf{1}[r_u \le 10]}{\log_2(r_u + 1)}, \tag{7}$$

$$\text{MRR@10} = \frac{1}{|U|} \sum_{u \in U} \frac{\mathbf{1}[r_u \le 10]}{r_u}. \tag{8}$$

With one relevant item the ideal DCG is 1. All metrics are computed per user, and per-user values serve directly as paired observations.

a) Beyond-accuracy metrics: Catalogue coverage@10 is the proportion of rankable items appearing in at least one user's top-10 list. The *top-10 share* is the fraction of all recommendation slots occupied by the ten most frequently recommended items. Intra-list diversity (ILD@10) averages, over users, the mean pairwise cosine dissimilarity between the category vectors of the ten recommended items. The category vectors are one-hot over first-level categories. Second-level categories and the multi-hot KuaiRand `tag` field (46 distinct tags among KuaiRand-Pure videos) serve as sensitivity checks.

b) Category and caption data: Categories and captions come from two supplementary files released with KuaiRand [16] (https://zenodo.org/records/18159199): a model-predicted four-level category hierarchy and each video's (Chinese) caption text. Of the 7,210 items, 99.83% have a first-level category (38 categories), 85.46% a second-level category (154), and 98.90% a non-empty caption. Alignment with KuaiRand-Pure was checked in two ways. The first compares recorded durations (Section V-D). The second uses category co-occurrence. Consecutive clicks by the same user within 30 minutes share a first-level category 1.89 times as often as independence predicts, against 0.985 ± 0.018 under permutation (200 permutations).

F. Training and Hyperparameters

All models are trained with Adam for at most ten epochs, with a causal history window of $L = 20$. Training stops early once validation NDCG@10 has not improved for three epochs (patience three). All reported comparisons use matched default hyperparameters, including a batch size of 512 and a learning rate of 10^{-3}. Only the CAFREC-specific profile and gating dimensions are fixed a priori, and no model in these comparisons receives a larger tuning budget than any other. The absolute values are those of this untuned default operating point. A partial search over SASRec and CAFREC-H, run after these results, is summarised in Section VI-G. A search that gives SASRec and CAFREC-NP an equal budget is reported in Section VI-B. The reported runs predate the harness change that writes the resolved configuration into every result file. Their batch sizes are established from the versioned launch code. A manifest in the released repository,

TABLE II
COMPOSITION OF THE CONTEXT VECTOR x_{CTX} AND ITS KUAIRAND SOURCES. ALL FEATURES ARE COMPUTED ON THE SESSION PREFIX.

Field	Role	KuaiRand source	Definition
`prefix_session_len_log_z`	core	`time_ms` (30-min gap)	z-scored $\log(1+\cdot)$ number of interactions so far in the session
`prefix_dwell_entropy_z`	core	`play_time_ms`, `duration_ms`	z-scored base-2 entropy of the dwell ratio `play_time_ms`/`duration_ms` over a 10-bin histogram
`prefix_category_drift_z`	core	`tag` (`video_features_basic`)	z-scored number of changes in the video's primary (first-listed) tag between consecutive prefix items, divided by $\max(\text{prefix length} - 1,\ 1)$. Exactly 0 before z-scoring for prefixes of length 0 or 1, the case for the 68.5% of held-out interactions that open a session
`prefix_policy_flag`	core	`is_rand`	Binary indicator of a within-session transition between standard and random policy
`inter_session_gap_log_z`	auxiliary	`time_ms`	z-scored log gap since the user's previous session
`is_first_session`	auxiliary	`time_ms`	Binary indicator of the user's first session

`cafrec_harness/results/MANIFEST.csv`, maps each reported value to its result file, table row, commit, launch script, batch size, seeds, and device.

a) Batch size: Under a fixed epoch budget, batch size sets the number of optimiser steps. An earlier configuration trained SASRec at batch size 2048 and CAFREC at 512, so SASRec took a quarter of the optimiser steps per epoch. The harness set batch sizes by a string test on the dataset name, which sent the context-augmented dataset down the large-catalogue branch. The result files recorded model, seed, and metrics but no hyperparameters, so the mismatch was invisible on disk. It first showed up as a disagreement between a local and a cloud run of one configuration, which was at first misread as a hardware effect. The mismatch alone moved SASRec's HR@10 by 2.9% (0.0787 versus 0.0810). It also produced an apparent 3.7% HR@10 advantage for context-gated SASRec, with the same sign in all seven seeds. Encoder, loss, data, and every architectural hyperparameter were matched. The pooled per-user p-value was 4.5×10^{-4}. In exploratory seed-level tests (Table III), the exact Wilcoxon p was 0.016 and the paired t-test p was 0.0015. With the batch size matched, the advantage falls to $+0.7\%$ (Section VI-A). Section V-D describes the fix, and Section VII-A what the case implies for evidence. Every comparison reported here uses batch size 512 unless marked.

b) Equal-budget tuning: Because the primary comparison uses matched defaults, a separate search gives SASRec and CAFREC-NP the same budget. Each model was searched over the same 16 configurations, crossing four factors at two levels each. Dropout was $\{0.5, 0.2\}$, applied to both hidden and attention layers, and depth was $\{2, 4\}$, with the number of heads tied to the number of layers. Learning rate was $\{10^{-3}, 3 \times 10^{-3}\}$ and weight decay $\{0, 10^{-5}\}$. The grid contains the default configuration and the configuration selected by the earlier partial search. Batch size, epoch budget, history window, loss, and data are fixed across the grid, so within it the two models differ only by the gate. Each configuration was trained once at seed 403092, which is not one of the seven evaluation seeds. The configuration with the highest *validation* NDCG@10 was then retrained at all seven. Test data played no part in selection.

c) Best validation epoch: The epoch at which validation NDCG@10 peaked is known only for the runs trained locally, from their saved checkpoints. The result files of the A10G runs, the headline runs among them, record the best validation score but not its epoch. CAFREC-NP at the default configuration peaks at epoch 9 or 10 of 10 in all three CPU runs. So does every one of the 24 search runs at seed 403092 that did not diverge (Section VI-B), 12 configurations per model with the default among them. At the selected configurations, retrained over the seven evaluation seeds, the best epoch ranges from 5 to 10. The medians are 7 for SASRec and 8 for CAFREC-NP. Three of the 14 runs peak at epoch 10.

G. Statistical Analysis

The headline comparison between SASRec, CAFREC-NP, CAFREC, HGN, and U-GRU uses seven seeds $\{42, 77, 123, 256, 512, 1024, 2048\}$. Ablations, controls, and sensitivity analyses use three seeds $\{2020, 2021, 403092\}$. Every comparison is paired over the $22{,}912$ test users. Each user's metric is averaged over the seeds shared by both conditions, and per-user differences are tested with the Wilcoxon signed-rank test. A 95% confidence interval for the mean difference is computed with a paired bootstrap ($2{,}000$ user resamples, resampled jointly across both models). Tests of a pre-specified hypothesis within its partitions are corrected for multiple comparisons by the Holm–Bonferroni method, within that hypothesis's family of tests.

a) Choice of tests: Two sources of variation matter here, and they answer different questions. Pairing over users estimates the quantity of interest, the difference in accuracy a user would experience. It uses the $22{,}912$ paired observations that give the comparison its resolution. Pairing over seeds asks whether the difference survives retraining. A pooled per-user test can reach significance on a difference that one training run reverses. Every comparison therefore also reports the number of seeds reaching $p < 0.05$ (*Sig.* in the tables). It also reports the number whose mean difference shares the sign of the pooled difference (*Sign*). Comparisons over at least five seeds add a paired test over the per-seed means. Where the two disagree, both are given and the more conservative one is

reported, whichever it is. No claim rests on the smaller p-value.

Most per-user HR@10 differences are exactly zero. The metric is an indicator, and for most users both models place the held-out item on the same side of the top-ten cut-off. Because the signed-rank test ties on those pairs, it addresses a different quantity from the mean-difference intervals. Inference below rests on the bootstrap intervals and the seed-level tests; the Wilcoxon p-values are secondary. Seven seeds is the largest number this budget allowed for the headline comparison. At that number, the smallest p an exact two-sided sign-based test can reach is 0.016. That suffices to detect a consistent direction, but not to estimate the seed variance precisely. A model with crossed user and seed random effects would use both sources at once, but it was not specified in the plan. The pairing above and the per-seed tests are reported instead.

Determinism is partial. Every run sets the framework's reproducibility switch, which seeds Python, NumPy, and PyTorch and sets `cudnn.deterministic` true and `cudnn.benchmark` false. However, `torch.use_deterministic_algorithms` was never called, so an operation with a non-deterministic default kernel could still vary between repetitions of one seed.

The content-bearing profiles and their CAFREC-NP reference were trained on a CPU at the three ablation seeds. These runs are paired only with each other. CPU and GPU runs of the same configuration and seed are not numerically identical: HR@10 differed by 1.9% for CAFREC-NP at seed 42 and batch size 2048. Their primary comparison is the dense cohort, with Holm–Bonferroni adjustment over three profiles and two metrics. The one comparison that crosses devices is reported as indicative only.

H. Status of Each Analysis

Table III classifies every analysis in the paper by when it was specified. *Pre-specified* means stated in the project plan submitted to the university in May 2026, before any experiment was run. The plan was not deposited in a public registry. That plan states H1 to H3. H4 was added in an August 2026 revision, after the first single-seed development runs. The hypotheses are quoted from that revision. H1's baseline list also changed between the two versions, from HRNN and SASRec to HGN, SASRec, and HGRU4Rec. H4 is therefore reported as added after the first results, not as pre-specified. Analyses *added after the first results* are controls or extensions designed once at least the first aggregate comparison was known. *Exploratory* analyses were defined after the aggregate result was known, or selected on the test split. They are hypothesis-generating and require confirmation before any claim rests on them.

Each condition named in Section VI is a configuration setting on the same model class. It differs from full CAFREC only in the selected branch of the forward pass, never in data, training schedule, or evaluation. It carries the same name in the released harness. Table IV lists each condition with what it changes, what it is paired against, its seeds, and its status.

Two further departures bear on how a result should be read. H2 names LLM-generated profiles, whereas the plan's implementation specified a templated summary embedded by a pretrained encoder. Both were built, and Section VI-C3 reports the pre-specified test. The pre-specified baseline comparison keeps HGN and U-GRU at their native pairwise loss, and the cross-entropy variants were added after the first results. Every pre-specified ablation was run. However, four gaps remain, two of them from the original plan. Individual context features were not ranked by retraining, and the equal-budget search covers only SASRec and CAFREC-NP. No KuaiRand-1K result is reported, and the content-bearing profiles were not extended to the seven headline seeds.

TABLE III
STATUS OF EACH ANALYSIS. P: PRE-SPECIFIED IN THE PROJECT PLAN; A: ADDED AFTER THE FIRST RESULTS; E: EXPLORATORY (POST HOC)

Analysis	Status	Sec.
CAFREC-NP and CAFREC vs SASRec, all users	P	VI-A
CAFREC vs HGN and U-GRU, native loss	P	VI-E
Profile removal (CAFREC vs CAFREC-NP)	P	VI-C
Sparse and dense cohorts	P	VI-C1
Static gate, concatenation, four-feature ablations	P	VI-D
Policy-flag ablation (five-feature x_{ctx})	P[f]	VI-B
H3 within activity and drift strata	P[a]	VI-D
H4 coverage and ILD within drift strata	A[g]	VI-F
HGN and U-GRU under cross-entropy	A	VI-E
Equal-budget tuning, SASRec and CAFREC-NP	A[d]	VI-B
Trainable stand-in, 7B encoder, hashing control	A	VI-C
Content-bearing profiles, dense cohort	A[b]	VI-C2
LLM-generated profile, lowest quartile (H2)	A[e]	VI-C3
Context-free vector gate; shuffled context, three and seven seeds	A[h]	VI-D
History-window sensitivity, not reported	A[i]	III-B
Session position, incl. H1's short-session clause	E	VI-G
Shuffled context by session position	E	VI-G
Shuffled context, pooled ten-seed estimate	E	VI-D
Minimum detectable effect (sensitivity power analysis)	E	VI-A
Seed-level tests of the batch-size mismatch	E	IV-F
Constant-profile control	E	VI-G
Gate values by position, drift, and dimension	E	VI-G
Within-position feature permutation at test time	E	VI-G
Timeline-leak query	E	VII-D
History length as a gate input (CAFREC-H)	E[c]	VI-G
Partial grid search (SASRec, CAFREC-H)	E[c]	VI-G

[a] Hypothesis pre-specified; strata fixed after the aggregate results, before any stratified result. [b] Primary test stated before the profiles were built; three seeds, CPU. [c] Selected from three variants on the test split. [d] Search space, selection rule, and evaluation seeds fixed before the runs; selection on validation data at a seed not used for evaluation. [e] Hypothesis pre-specified; the generator, prompt design, encoder, and comparison set were fixed before the profiles were generated; three seeds. [f] Pre-specified in the project plan; run last, at the tuned operating point, over seven seeds. [g] H4 was added in the August 2026 revision, after the first development runs; strata fixed after the aggregate results, and ILD computed after the accuracy results. [h] The shuffled control was extended from headline seeds 42, 77, and 123 to all seven after that three-seed result was known; the ablation seeds were run last. [i] Confounded by batch size (SASRec at 2048, CAFREC variants at 512).

V. SYSTEM DESIGN AND IMPLEMENTATION

A. Harness Architecture

Fig. 2 shows the path from the raw log to a reported value. Each stage writes a versioned artefact that the next stage reads, so a stage can be rerun or replaced without touching the others. Each stage at which information could cross the boundary between training and test data is guarded by tests. Sessions are cut on the full impression log at a 30-minute

TABLE IV
CONDITIONS COMPARED IN SECTION VI. SEEDS: 7, THE HEADLINE SET; 3, THE ABLATION SET (SECTION IV-G). STATUS AS IN TABLE III

Condition	What it changes	Paired against	Seeds	Status
Models				
SASRec	Reference encoder; the $g \to \mathbf{0}$ limit of CAFREC	–	7	P
CAFREC-NP	Adds the context gate to SASRec; no profile	SASRec	7	P
CAFREC	Adds the templated profile to CAFREC-NP	SASRec; CAFREC-NP	7	P
HGN, U-GRU	Hierarchical baselines, under BPR and cross-entropy	SASRec; CAFREC-NP	7	P; A
Tuned SASRec, CAFREC-NP	Equal-budget 16-point search	Each other	7	A
CAFREC-H	Adds log history length to the gate input	SASRec; CAFREC-NP	7	E
Fusion variants and gate controls				
Static gate	One learned scalar replaces the context gate	CAFREC	3	P
Concatenation	Projection of $[\, z_{\text{long}}; z_{\text{short}} \,]$ replaces the gate	CAFREC	3	P
Four-feature x_{ctx}	Drops the gap and first-session indicator	CAFREC	3	P
Five-feature x_{ctx}	Drops the policy flag; tuned configuration	CAFREC-NP, tuned	7	P
Vector gate	$g = \sigma(\theta)$, no input	CAFREC-NP	3	A
Shuffled context	x_{ctx} permuted across sessions	CAFREC-NP	7; 3	A
Long-term profiles				
Stand-in	Trainable per-user table replaces the frozen profile	CAFREC	3	A
Hashing	CRC32 bag-of-tokens embedding of the template	`bge-large` profile	3	A
7B	Template encoded with `e5-mistral-7b`	`bge-large` profile	3	A
`cat_beyond`	Category histogram, beyond-window rows	CAFREC-NP (CPU)	3	A
`cap_beyond`	Mean caption embedding, beyond-window rows	CAFREC-NP (CPU)	3	A
`cap_all`	Mean caption embedding, all training rows	CAFREC-NP (CPU)	3	A
Generated	`Qwen2.5-7B` paragraph, `bge-large` encoded	Stand-in	3	A
Constant	Corpus-mean profile for every user	Stand-in	3	E

gap. The context features are computed on each session's prefix and z-scored with a scaler fitted on training rows only. The scaler is persisted and applied unchanged to validation and test rows. The profile builders read only training rows and write one frozen tensor in RecBole's internal user order. The model registry described below then trains every model through the same loader, splitter, and trainer. After training, a full-ranking module recovers each user's rank of the held-out item, and the user's top-10 list, under the same masking RecBole applies internally. Both are keyed by the *original* user identifier. That key lets a model trained on the plain dataset be paired user-for-user with one trained on the context-augmented dataset. Metrics aggregated from the dumped ranks reproduce RecBole's reported metrics for every condition. The statistics module reads only these dumps, and the manifest of Section IV-F maps every reported value to its result files.

All models are implemented in one harness built on RecBole [46] and PyTorch [47]. Each model is a single specification entry naming a RecBole built-in (SASRec, HGN) or a local class (CAFREC, U-GRU) together with its loss contract.

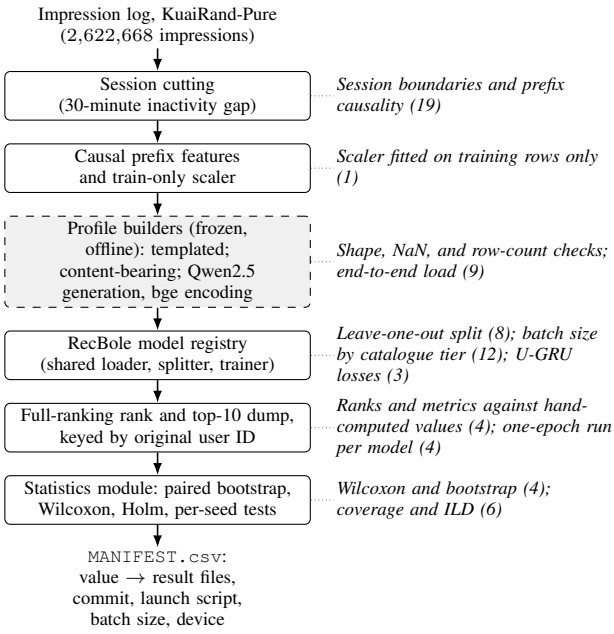


Fig. 2. Experimental pipeline from interaction log to reported value. Each stage writes an artefact the next stage reads. Italic notes give the tests that guard each stage, with their counts; the suite holds 70 tests in all. Dashed: frozen component built offline.

All models share data loaders, the leave-one-out splitter, full-catalogue ranking, and checkpoint format by construction. U-GRU is not a RecBole built-in. It is implemented as a session-level GRU over the recent-item window seeded by a learnable per-user state. This approximates cross-session state propagation, because the loader supplies no in-window session delimiters. Every result reported for U-GRU therefore concerns this harness implementation, and is not evidence about the published performance of the architecture of Quadrana et al. [3]. The same caution applies in weaker form to every baseline: each runs at the defaults of this harness rather than at an author-tuned configuration.

B. Test Suite

The suite holds 70 tests in nine files, and all 70 pass. Grouped by the failure they guard against, 8 protect split integrity. They check that leave-one-out targets follow time rather than file order, that sampled negatives exclude seen items, and that the split is reproducible. A further 19 protect prefix causality. They check that sessions are cut strictly at the 30-minute gap and that each feature is defined on the prefix alone. They also check that appending a row leaves every earlier row unchanged, and that a held-out target sees only its own prefix. Scaler fitting has 1 test, which checks that the scaler is fitted on training rows only. Profile row alignment is protected by 9, which cover shape, dtype, and NaN validation, rejection of extra rows, zero-filling of a missing tail, and an end-to-end load into CAFREC. Full ranking is covered by 8: 4 check ranks and metrics against hand-computed values, and 4 run one epoch of each model end to end. The remaining 25 cover the batch-size defaults (12), U-GRU's losses (3), the statistics module (4), and the diversity metrics (6).

C. Compute

Three devices were used. Comparisons are paired within a device, and the one that crosses devices is reported as indicative only (Section IV-G). Every training run not named below ran on NVIDIA A10G GPUs rented through Modal. These include the headline runs, the baselines, the controls, and the four H2 conditions (Section VI-C3). Runs were launched in parallel. A sweep of 14 jobs finished in 314 to 545 s of wall-clock time, and the measured cost was \$0.42 to \$0.61 per job. The content-bearing profiles ran on a four-core Intel Xeon 8272CL virtual machine at about 41 min per ten-epoch run. The equal-budget search and its retraining ran on one NVIDIA RTX 2060 SUPER, three to five runs at a time, at about 31 min per run. The 53 runs took 7 h 13 min of wall-clock time. Training on that card was bound by CPU-side dispatch, about 150 ms per batch against 11 ms of GPU work.

D. Design Decisions

Three decisions in the harness each caught, or ruled out, a defect that would otherwise have reached the results.

Joining supplementary data through an independent field. Matching identifiers do not prove matching videos. A reserved padding index or a mixed 0- and 1-based convention can offset an index by one. That error would attach every category and caption to the wrong video and still yield plausible output. Before the category and caption files were used, their recorded durations were compared with KuaiRand-Pure's. They agree for all 6,918 videos that have both, and for none when identifiers are offset by one (Section IV-E). The check found the join correct, and it fails loudly if a rebuild shifts the index.

Writing the resolved configuration into every result file. A string test on the dataset name once set batch sizes, and result files recorded no hyperparameters. Together these produced and hid the batch-size mismatch of Section IV-F. The fix keys batch defaults to the catalogue tier instead of the dataset name, raises an error for an unknown dataset, and writes the resolved configuration into every result file. It is covered by 12 tests, and the manifest reconstructs the setting for runs that predate it.

Recomputing every metric from dumped ranks. Because the rank dump reproduces RecBole's metrics, it gives each run an independent check. On KuaiRand-1K one baseline scored a test HR@10 of zero. The recomputation agreed, which ruled out a fault in evaluation and pointed to the data. The 1K file had no item floor, and 33.5% of its held-out targets occur only once in it. They therefore never appear in training, and no model can rank them. That finding is why the larger tiers need a k-core floor before use (Section IV-B).

VI. Results

All results are from KuaiRand-Pure under leave-one-out splitting and full-catalogue ranking. Every model is trained at batch size 512 unless marked. Table III gives the status of each analysis. Exploratory results are marked where they occur and collected in Section VI-G. Differences and percentages are computed from unrounded values, so they can differ from values recomputed from the rounded table entries.

TABLE V
Top-10 Accuracy on KuaiRand-Pure Under Full-Catalogue Ranking (Mean ± Standard Deviation Across Seven Seeds; All Models at Training Batch Size 512). Tuned Rows Ran on One RTX 2060 SUPER, All Other Rows on A10G GPUs

Model (loss)	HR@10	NDCG@10	MRR@10
SASRec (CE)	0.0810 ± 0.0015	0.0411 ± 0.0009	0.0291 ± 0.0007
CAFREC-NP (CE)	0.0816 ± 0.0014	0.0416 ± 0.0006	0.0297 ± 0.0006
CAFREC (CE)	0.0810 ± 0.0016	0.0415 ± 0.0008	0.0297 ± 0.0006
SASRec, tuned	0.0880 ± 0.0018	0.0456 ± 0.0008	0.0329 ± 0.0006
CAFREC-NP, tuned	0.0888 ± 0.0008	0.0462 ± 0.0002	0.0335 ± 0.0003
HGN (CE)	0.0717 ± 0.0014	0.0340 ± 0.0007	0.0227 ± 0.0005
U-GRU (CE)	0.0668 ± 0.0010	0.0314 ± 0.0004	0.0209 ± 0.0004
HGN (BPR)	0.0235 ± 0.0023	0.0112 ± 0.0008	0.0074 ± 0.0005
U-GRU (BPR)	0.0554 ± 0.0026	0.0278 ± 0.0013	0.0195 ± 0.0010

A. Context-Gated SASRec Versus SASRec

Context-gated SASRec (CAFREC-NP) does **not** significantly outperform SASRec (Tables V and VI). Because CAFREC-NP differs from SASRec only by (2), this is the study's most direct test of session-context gating. Its mean advantage is $+0.7\%$ in HR@10, $+1.4\%$ in NDCG@10, and $+1.9\%$ in MRR@10. No pooled test survives Holm correction (smallest $p_{\text{Holm}} = 0.27$). The HR@10 difference has the same sign in only four of seven seeds, and no seed reaches significance on HR@10 or NDCG@10. CAFREC is indistinguishable from SASRec on HR@10 ($\Delta = +0.0000$, $p = 0.90$). With both models at batch size 2048 the difference is also null ($+0.2\%$ HR@10, $p = 0.85$). These results are a failure to detect an effect, not evidence of no effect. The NDCG@10 and MRR@10 intervals lie mostly above zero, and a gain of up to about 2% in HR@10 remains compatible with the data (below).

H1 is therefore **not supported** against SASRec. CAFREC-NP is significantly more accurate than HGN and U-GRU under both losses (Section VI-E). So is SASRec, so that margin comes from the self-attentive encoder, not from context gating.

At the default operating point, the seed-to-seed standard deviation of HR@10 is between 0.0014 and 0.0016 for SASRec, CAFREC-NP, and CAFREC (Table V). That is more than twice the mean difference between SASRec and CAFREC-NP. Fig. 3 shows the per-seed differences behind that mean. Depending only on the seed, the same pair of models on the same data and split produces differences from -1.3% to $+2.9\%$. The seed-to-seed standard deviation of the difference is 1.7% of SASRec's HR@10, more than twice the mean effect. Under a normal approximation, a single pair of runs would show the opposite sign to the seven-seed mean about one time in three.

Two intervals bound the gate's possible HR@10 benefit at about 2% of SASRec's HR@10. The seven per-seed differences have mean $+0.00061$, SD 0.00138, and standard error 0.00052. On six degrees of freedom this gives a 95% interval of $[-0.00066, +0.00189]$ and a paired t-test p-value of 0.283. The seed-level 95% interval agrees with the paired user bootstrap, $[-0.0006, +0.0018]$. Relative to SASRec's HR@10 of 0.0810, the seed-level interval runs to $+2.3\%$ and the bootstrap interval to $+2.2\%$. The bound quoted elsewhere, about 2%, is

TABLE VI
PAIRED PER-USER COMPARISONS AGAINST SASREC (SEVEN SEEDS, BATCH SIZE 512). Δ IS THE MEAN PER-USER DIFFERENCE AND THE CI A PAIRED BOOTSTRAP 95% INTERVAL. p_{HOLM} IS HOLM–BONFERRONI-ADJUSTED OVER THE SIX TESTS IN THIS TABLE. *Sig.* AND *Sign* AS IN SECTION IV-G.

Comparison	Metric	Δ	95% CI	p	p_{Holm}	Sig.	Sign
CAFREC-NP vs SASRec	HR@10	$+0.0006$	$[-0.0006, +0.0018]$	0.45	1.00	0/7	4/7
	NDCG@10	$+0.0006$	$[-0.0000, +0.0012]$	0.11	0.54	0/7	6/7
	MRR@10	$+0.0006$	$[-0.0000, +0.0011]$	0.045	0.27	2/7	6/7
CAFREC vs SASRec	HR@10	$+0.0000$	$[-0.0016, +0.0015]$	0.90	1.00	0/7	3/7
	NDCG@10	$+0.0005$	$[-0.0003, +0.0012]$	0.34	1.00	0/7	6/7
	MRR@10	$+0.0006$	$[-0.0001, +0.0013]$	0.23	0.91	1/7	7/7

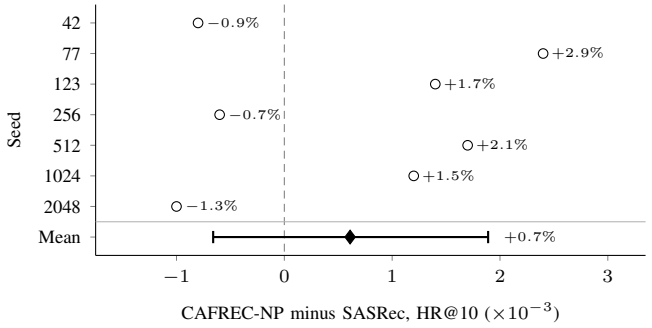


Fig. 3. Per-seed differences behind the primary comparison at batch size 512. Circles: CAFREC-NP minus SASRec in HR@10 for each seed, rounded to four decimals, labelled with the difference as a percentage of SASRec's HR@10 in that seed. Diamond and bar: the mean difference and its seed-level 95% interval, $[-0.00066, +0.00189]$ (t distribution, six degrees of freedom).

the round figure the two share. Both intervals lie inside $\pm 2.9\%$, the shift that the batch-size mismatch of Section IV-F alone produced in SASRec's HR@10. At this operating point the gate's effect on HR@10 is therefore smaller than that artefact. An exploratory sensitivity analysis gives the resolution of the design. At the observed seed-level standard deviation, seven seeds detect a difference of 2.2% of SASRec's HR@10 with 80% power at $\alpha = 0.05$. That standard deviation rests on six degrees of freedom, and its 95% interval places the minimum detectable effect between 1.4% and 4.8%. This is a sensitivity power analysis of the design, not observed power [48].

B. Equal-Budget Tuning

An equal tuning budget leaves the null unchanged. At the configurations selected for each model, which differ in depth, the gate still does not significantly improve on SASRec. The result above comes from a default operating point, so it could reflect an under-tuned gate rather than an ineffective one. Both models were therefore given the identical 16-point search described in Section IV-F, with selection on validation NDCG@10 at a seed not used for evaluation.

The selected configurations differ. SASRec attains its best validation NDCG@10, 0.0496, with dropout 0.2, four layers and four heads, learning rate 10^{-3}, and weight decay 10^{-5}. Within the four factors varied here, the earlier partial search selected the same configuration. CAFREC-NP attains its best, 0.0507, with dropout 0.2, two layers and two heads, learning rate 10^{-3}, and no weight decay. Its four-layer configuration is close behind at 0.0502. For both models, dropout 0.2 improves on 0.5, as the earlier search found. A learning rate of 3×10^{-3} diverges at four layers for both (validation NDCG@10 between 0.012 and 0.028).

Tuning raises both models by about 11% in NDCG@10 relative to the default operating point, and leaves the comparison between them unchanged (Table V, tuned rows). Over all users the gate's advantage is $+1.0\%$ in HR@10 and $+1.4\%$ in NDCG@10 (95% CIs $[-0.0005, +0.0021]$ and $[+0.0000, +0.0013]$; across-seed $p = 0.35$ and 0.072). It has the same sign in five of seven seeds. No test survives Holm adjustment over the six tests of this comparison (all users, openers, and within sessions, on two metrics). Within sessions the intervals are 1.8 and 1.9 times as wide as over all users, as the cohort's size predicts (7,206 of 22,912 users). The within-session null is the least precise of the three. Each configuration was selected from one seed, and the selected configurations differ in depth. The search therefore makes under-tuning a less likely explanation without ruling it out.

The tuning gain of about 11% is not a paired estimate, because it compares two operating points measured on different hardware. One configuration was run on both. CAFREC-NP at the default configuration and seed 403092 reaches validation NDCG@10 0.0451 on the RTX 2060 SUPER and 0.0452 on the A10G. Its test NDCG@10, however, differs by 4% (0.0417 against 0.0400). Comparisons between models are paired within a device throughout, and the paired differences above are unaffected.

a) Policy-flag ablation: Removing `prefix_policy_flag` leaves a five-feature x_{ctx}. This ablation was pre-specified in the project plan and run last, at the tuned operating point, over seven seeds. Neither all-user change is significant. Over all users the removal changes HR@10 by -0.8% (95% CI $[-0.0017, +0.0003]$, $p_{\text{Holm}} = 0.72$) and NDCG@10 by -0.0% (95% CI $[-0.0005, +0.0005]$, $p_{\text{Holm}} = 0.83$). The changes are -1.0% and -0.1% for openers, and -0.4% and $+0.0\%$ within sessions. The all-user intervals exclude changes larger than 1.9% of the reference. The flag marks a prefix that contains a transition between the organic and random-exposure policies, and is set for 0.98% of target rows (Section IV-C).

C. Contribution of the Long-Term Profile

No long-term profile improves on the model without one in a primary test. The paper counts five long-term profiles:

TABLE VII
PAIRED TESTS ISOLATING THE PRE-SPECIFIED LONG-TERM BRANCH. PROFILE REMOVAL WAS PRE-SPECIFIED; THE OTHER COMPARISONS WERE ADDED AFTER THE FIRST RESULTS

Comparison	Metric	Δ	95% CI	p
CAFREC vs CAFREC-NP (7 seeds)				
	HR@10	-0.0006	$[-0.0019, +0.0007]$	0.54
	NDCG@10	-0.0001	$[-0.0008, +0.0005]$	0.78
e5-mistral-7B vs bge-large profile (3 seeds)				
	HR@10	-0.0005	$[-0.0018, +0.0008]$	0.21
	NDCG@10	-0.0005	$[-0.0011, +0.0002]$	0.21
Hashing vs bge-large profile (3 seeds)				
	HR@10	$+0.0004$	$[-0.0009, +0.0017]$	0.60
	NDCG@10	$+0.0003$	$[-0.0004, +0.0009]$	0.46
Frozen bge profile vs trainable stand-in (3 seeds)				
	HR@10	$+0.0077$	$[+0.0053, +0.0098]$	6.5×10^{-15}
	NDCG@10	$+0.0054$	$[+0.0041, +0.0067]$	6.5×10^{-11}

the pre-specified templated profile, the three content-bearing profiles, and the generated profile. The 7B-encoded and hashing variants re-encode the templated text and are counted with it. Table VII isolates the pre-specified long-term branch, whose removal was pre-specified. Replacing it with a trainable embedding of identical size, and changing its text encoder, were added after the first results.

Adding the template profile to context-gated SASRec produces no measurable change: CAFREC does not differ from CAFREC-NP, and both intervals include zero. Replacing the 335M-parameter text encoder with the 7B-parameter encoder produces no measurable change either. A hashed bag-of-tokens embedding of the same text, which captures no meaning, performs indistinguishably from `bge-large` ($p = 0.60$ on HR@10). The profile is a user-level average of the behavioural statistics the gate already receives per session (Section III-C). It restates information the model has in a more timely form. The experiments do not test whether this redundancy causes the null.

The frozen profile outperforms a randomly initialised *trainable* per-user table of the same size by 0.0077 in HR@10. A frozen hashing profile with no semantic content does as well. The gap therefore lies in the freezing of the per-user table rather than in profile content.

1) History Cohorts: In neither pre-specified cohort does any interval exclude zero, for the profile or for the gate. The largest difference is CAFREC-NP over SASRec in the dense cohort, $+0.0012$ HR@10 (95% CI $[-0.0001, +0.0027]$). Sparse users are the easier group under this protocol. SASRec's HR@10 is 0.1059 for sparse users and 0.0604 for dense users, and CAFREC-NP and CAFREC show the same gap. The twenty-item window covers most of a sparse user's history but truncates a dense user's. Only for dense users can a long-term branch carry information the encoder does not see, and Section VI-C2 tests that case.

2) Content-Bearing Profiles: No content-bearing profile improves dense-cohort accuracy, and the two caption profiles lower it significantly. Table VIII compares the three profiles, added after the first results, with CAFREC-NP trained on the same CPU at the same three seeds. In the dense cohort, the primary test stated before the profiles were built, all three profiles lower HR@10 and NDCG@10 in all three seeds. The losses are significant after adjustment for `cap_beyond` (-6.5% and -5.8%) and `cap_all` (-19.3% and -22.3%), but not for `cat_beyond` ($p_{\text{Holm}} = 0.13$). One exploratory result runs the other way: in the lowest quartile, `cap_beyond` exceeds CAFREC-NP by 4.0% NDCG@10 ($+0.0024$, 95% CI $[+0.0003, +0.0045]$, $p = 0.022$). Every `cap_beyond` vector is zero in that quartile, and z_{long} reduces to the learned bias b_p. Any gain there comes from a constant vector rather than watched content. The finding that no profile improved on CAFREC-NP refers to the primary tests.

The caption profiles are close to one shared vector. The mean pairwise cosine between users' `cap_all` vectors is 0.85, and each lies at a mean cosine of 0.92 to the average profile. For `cat_beyond` the corresponding values are 0.40 and 0.63. With `cap_all`, validation NDCG@10 peaks at epoch 4 of 10 in all three seeds and training stops early. CAFREC-NP and the other two profiles peak between epochs 8 and 10. `cap_all` averages every training target into the profile (Section III-C). An early validation peak is consistent with fitting that signal and losing it at evaluation. The two explanations, a near-constant shared vector and a fitted target signal, were not separated. Against the CPU-trained CAFREC-NP, `cap_all` raises coverage (0.253 against 0.236) and lowers ILD (0.759 against 0.811). `cat_beyond` and `cap_beyond` raise ILD to 0.824 and 0.827, with coverage close to that of CAFREC-NP.

3) LLM-Generated Profiles (H2): H2 is inconclusive. In the lowest activity quartile the generated profile does not outperform the trainable stand-in, but an unmeasured training-target leak prevents reading this as a failure of the hypothesis. H2 predicted that LLM-generated temporal profiles would achieve higher NDCG@10 than interaction-derived embeddings in the lowest activity quartile. That quartile holds the $6{,}240$ users with at most 12 organic clicks. H2's interaction-derived embedding is operationalised as the trainable per-user stand-in. That choice was fixed with the rest of the comparison set before the profiles were generated (Table III, note e). Four conditions were trained on one code path at batch size 512 over the three ablation seeds, so they are mutually comparable. They are the generated profile, the templated profile, the stand-in, and CAFREC-NP. Holm adjustment covers the four lowest-quartile tests in this family. Every user in the quartile carries a generated profile, unlike the `cap_beyond` case, in which the cohort held zero vectors.

The generated profile changes lowest-quartile NDCG@10 by -0.4% relative to the stand-in ($p_{\text{Holm}} = 0.80$, the same sign in two of three seeds). The paired interval over users is $[-0.0032, +0.0027]$ against a stand-in base of 0.0514. It excludes gains larger than about 5% of the stand-in's accuracy in that quartile. The predicted advantage is not observed. The label is inconclusive rather than not supported because of the training-target leak (Section III-C). The leak is expected to work against the generated profile and was not measured, so this test cannot show whether the profile would help without it. Two results limit how far the leak can explain the null. The leak is equally complete in the second quartile, where the generated profile gains 6.2% NDCG@10 over the stand-

TABLE VIII
CONTENT-BEARING PROFILES AGAINST CAFREC-NP TRAINED ON THE SAME CPU, DENSE COHORT ($n = 12{,}567$; THREE SEEDS, BATCH SIZE 512; ADDED AFTER THE FIRST RESULTS). p_{HOLM} IS ADJUSTED OVER THREE PROFILES AND TWO METRICS. *Sig.* AND *Sign* AS IN SECTION IV-G.

Profile	Metric	Δ (%)	95% CI	p	p_{Holm}	Sig.	Sign
cat_beyond	HR@10	-0.0023 (-3.8)	$[-0.0045, -0.0002]$	0.065	0.13	0/3	3/3
	NDCG@10	-0.0011 (-3.5)	$[-0.0021, -0.0001]$	0.068	0.13	1/3	3/3
cap_beyond	HR@10	-0.0040 (-6.5)	$[-0.0069, -0.0011]$	8.4×10^{-3}	0.034	1/3	3/3
	NDCG@10	-0.0018 (-5.8)	$[-0.0033, -0.0003]$	0.014	0.043	1/3	3/3
cap_all	HR@10	-0.0118 (-19.3)	$[-0.0152, -0.0087]$	1.2×10^{-10}	6.1×10^{-10}	3/3	3/3
	NDCG@10	-0.0068 (-22.3)	$[-0.0085, -0.0051]$	1.6×10^{-12}	9.6×10^{-12}	3/3	3/3

in. The gains are 10.5% and 6.7% in the third and fourth quartiles and 4.4% over all users. The advantage is therefore smallest in the quartile where H2 expected it to be largest.

Two comparisons in the family are larger than the pre-specified one, and both are negative. The generated profile falls 10.4% below the templated profile in the lowest quartile ($p_{\text{Holm}} = 4.6 \times 10^{-4}$, same sign in all three seeds). It falls 12.6% below CAFREC-NP, which carries no long-term branch at all ($p_{\text{Holm}} = 6.2 \times 10^{-7}$). HR@10 and MRR@10 agree in both cases: this generated profile costs accuracy for these users. The fourth test in the family reproduces the templated profile's advantage over the stand-in, at 11.1% ($p_{\text{Holm}} = 8.9 \times 10^{-5}$). The same contrast measured 10.6% on an earlier run set, which places the new conditions on the same scale as the profile results already reported.

The stand-in is a weak comparator. It is a randomly initialised, unregularised per-user table fitted to at most ten training rows per user in this quartile. The exploratory constant vector of Section VI-G exceeds it. The more informative reference in this family is CAFREC-NP, which the generated profile trails by the largest margin (above). Because the stand-in is weak, failing to exceed it counts against the generated profile rather than for it. A stronger interaction-derived comparator is left to future work (Section VII-E).

D. Context Conditioning and Gate Controls

The context gate outperforms both static alternatives, but the margins measure what the alternatives cost rather than a gain from context. The controls also do not establish that the gate uses session-specific context. A single learned scalar gate lowers NDCG@10 by 0.0019 and concatenation by 0.0054, in all three seeds (Table IX). Because the context gate does not itself improve on SASRec (Section VI-A), these differences measure the costs the alternatives impose on a SASRec representation.

Two controls on CAFREC-NP test whether the gate depends on a varying input and on the pairing of context with its own session. A context-free vector gate, $g = \sigma(\theta)$ with $\theta \in \mathbb{R}^H$, keeps the per-dimension freedom but has no input. It costs 3.7% in HR@10 ($p = 2.6 \times 10^{-4}$) and 0.0022 in NDCG@10, in all three seeds. Yet a constant per-dimension rescaling is already expressible *exactly* without a gate. The final Transformer block ends in a LayerNorm with learnable per-dimension gain γ and bias β, so $z_{\text{short}} = \gamma \odot \mathrm{LN}(x) + \beta$ and

$$(1 - g) \odot z_{\text{short}} = \big(\gamma \odot (1 - g)\big) \odot \mathrm{LN}(x) + \beta \odot (1 - g). \quad (9)$$

A constant g is therefore absorbed into γ and β. The 3.7% cost is an optimisation effect under the ten-epoch budget, not a loss of representational capacity. Optimisation alone can thus move accuracy by more than the gate effects under study. Concatenation starts further from SASRec than either gate, from a randomly initialised projection of $[\, z_{\text{long}}; z_{\text{short}} \,]$.

Permuting x_{ctx} across sessions, in training and evaluation, lowers accuracy by about 2% at most, and the evidence for even that is weak. On the seven headline seeds it costs 2.1% NDCG@10 (-0.0009, 95% CI $[-0.0015, -0.0003]$) and 1.8% HR@10 (-0.0014, $[-0.0027, -0.0002]$). Accuracy is lower in six of seven seeds. The pooled per-user p-values are 0.023 and 0.028, the exact across-seed Wilcoxon p-values 0.078 and 0.047, and the paired t-test p-values 0.040 and 0.032. Under the conservative rule of Section IV-G, the NDCG@10 loss is therefore not significant. HR@10 reaches $p = 0.047$, and no claim rests on that value. The control was first run on headline seeds 42, 77, and 123, where it lowered NDCG@10 in two of the three seeds. It was then extended to the other four headline seeds, paired with the CAFREC-NP runs of Table V. The seven-seed p-values are not adjusted for that look at three of the seeds (Table III, note h). On the three ablation seeds, run afterwards, the cost is 0.2% NDCG@10 (-0.0001, $p = 0.93$) and -0.0001 in HR@10 ($p = 0.81$). An exploratory pooled estimate, defined after both batches were known, covers all ten seeds. Pooled this way, the permutation changes NDCG@10 by -1.6% (95% CI $[-0.00131, +0.00002]$, $p = 0.084$) and HR@10 by -1.3% (95% CI $[-0.00208, -0.00001]$, $p = 0.11$). The split by session position is also exploratory and is reported in Section VI-G.

The control cannot localise the signal it removes. At an opener the prefix is empty, so the four core features take their empty-prefix values. The first-session indicator is 0 for every test case (Section VI-G). At test time the inter-session gap is therefore the only input that varies across openers. Shuffling supplies another session's context, usually with a non-empty prefix. It thereby removes the session-start signature and the opener's own gap along with the session's context. The control therefore cannot separate within-session context from a session-start or inter-session-gap signal. The mechanism H3 was meant to demonstrate is not established. Two further results bear on the gap. Retraining with x_{ctx} restricted to the four core features changes NDCG@10 by -0.00005 (95% CI $[-0.00071, +0.00068]$, $p = 0.74$). That restriction removes the gap and the first-session indicator but keeps the empty-prefix values that mark an opener. The run used three seeds with

TABLE IX
FUSION ABLATIONS (PRE-SPECIFIED) AND GATE CONTROLS (ADDED AFTER THE FIRST RESULTS), THREE SEEDS $\{2020, 2021, 403092\}$. FUSION VARIANTS KEEP THE PROFILE AND ARE PAIRED AGAINST CAFREC; GATE CONTROLS REMOVE IT AND ARE PAIRED AGAINST CAFREC-NP. Δ AND p REFER TO NDCG@10. *Sig.* AND *Sign* AS IN SECTION IV-G.

Variant	HR@10	NDCG@10	ΔNDCG	95% CI	p	Sig.	Sign
Fusion mechanism, profile present							
CAFREC (context gate)	0.0791 ± 0.0005	0.0405 ± 0.0004	–	–	–	–	–
Static scalar gate	0.0764 ± 0.0019	0.0385 ± 0.0009	-0.0019	$[-0.0028, -0.0011]$	6.3×10^{-6}	2/3	3/3
Concatenation	0.0722 ± 0.0019	0.0351 ± 0.0008	-0.0054	$[-0.0065, -0.0043]$	7.4×10^{-19}	3/3	3/3
Gate controls, profile removed							
CAFREC-NP (reference)	0.0799 ± 0.0012	0.0406 ± 0.0006	–	–	–	–	–
Context-free vector gate	0.0770 ± 0.0008	0.0385 ± 0.0002	-0.0022	$[-0.0029, -0.0015]$	2.8×10^{-9}	3/3	3/3
Shuffled context	0.0798 ± 0.0003	0.0406 ± 0.0006	-0.0001	$[-0.0009, +0.0007]$	0.93	1/3	1/3

the profile present, paired against CAFREC and unsplit by position. In an exploratory analysis, permuting the gap at test time in the trained CAFREC-NP gate costs 2.55% NDCG@10, the largest of the six features (Section VI-G). The first result measures whether a model trained without the gap can do as well. The second measures whether the trained gate relies on it, and both can hold. Section VII-B sets out the readings these results leave open. The shuffled model does not collapse to a constant gate, which is what the vector-gate control measures. Neither control was rerun at the tuned configuration.

a) Activity strata and drift: H3 as stated in the plan requires the context gate to win *across all activity strata*, with the largest margin on high-drift sessions. Table X reports NDCG@10 within each activity quartile and drift stratum, Holm-adjusted within the H3 family of 28 tests. Against concatenation, the context gate wins in every stratum, with every adjusted p-value below 0.05. Against the static scalar gate, the point estimate favours the context gate in all seven strata, but only the aggregate and the low-drift stratum survive adjustment. The quartile tests do not (p_{Holm} between 0.08 and 0.77).

The drift condition holds for one comparison only, and weakly. Against concatenation the margin is larger for high-drift sessions than for low-drift ones ($+0.0049$ in NDCG@10, 95% CI $[+0.0012, +0.0085]$). A Mann–Whitney test of the difference between strata, however, is not significant ($p = 0.10$). Against the static gate the margin runs the other way (-0.0015, $[-0.0042, +0.0011]$, $p = 0.098$). H3 is therefore **partly supported**: it holds against concatenation in every stratum, with the predicted drift ordering in the point estimate. Against the static gate it holds only in aggregate and in the low-drift stratum.

TABLE X
NDCG@10 DIFFERENCE BETWEEN THE CONTEXT GATE AND EACH STATIC FUSION ALTERNATIVE, BY ACTIVITY QUARTILE AND DRIFT STRATUM (THREE SEEDS). p_{HOLM} IS ADJUSTED WITHIN THE H3 FAMILY.

	vs static scalar gate		vs concatenation	
Stratum	**Δ (%)**	p_{Holm}	**Δ (%)**	p_{Holm}
All users	$+0.0019$ ($+4.9$)	1.2×10^{-4}	$+0.0054$ ($+15.4$)	2.1×10^{-17}
Q1 (≤ 12)	$+0.0019$ ($+3.4$)	0.18	$+0.0059$ ($+11.5$)	1.1×10^{-4}
Q2 (13–22)	$+0.0025$ ($+5.8$)	0.080	$+0.0077$ ($+19.9$)	8.2×10^{-9}
Q3 (23–38)	$+0.0020$ ($+6.3$)	0.22	$+0.0042$ ($+14.3$)	0.044
Q4 (> 38)	$+0.0012$ ($+5.5$)	0.77	$+0.0038$ ($+19.8$)	2.2×10^{-3}
High drift	$+0.0006$ ($+1.2$)	1.00	$+0.0096$ ($+24.2$)	2.9×10^{-5}
Low drift	$+0.0021$ ($+5.8$)	4.5×10^{-5}	$+0.0047$ ($+13.6$)	1.6×10^{-12}

E. Hierarchical Baselines

The self-attentive models outperform HGN and U-GRU under both losses, but matching the loss shrinks their lead. Table V reports HGN and U-GRU under two losses, at the same batch size and on the same seven seeds as the other models. Their native BPR loss is the pre-specified comparison, and SASRec's cross-entropy loss was added after the first results. The loss has a large effect. Switching to cross-entropy triples HGN's HR@10 ($+0.0482$, $p = 2.0 \times 10^{-219}$) and raises U-GRU's by 20.5% ($p = 3.9 \times 10^{-21}$), both in all seven seeds. It also reverses their order. The loss reverses comparisons between sequential architectures in the same way for SASRec and BERT4Rec [25]. BPR-trained HGN reaches HR@10 0.0235, below the 0.0432 of a most-popular recommender (Table XI). The pre-specified native-loss comparison against it is therefore weak evidence for H1.

With the loss matched, the self-attentive models remain the strongest. SASRec improves on HGN by 0.0093 in HR@10 ($+13.0\%$, 95% CI $[+0.0065, +0.0121]$, $p = 6.9 \times 10^{-10}$) and on U-GRU by 0.0142 ($+21.2\%$, $[+0.0117, +0.0166]$, $p = 1.7 \times 10^{-26}$). CAFREC-NP yields nearly identical margins ($+0.0099$ and $+0.0148$). Every one of these comparisons is significant and has the same sign in all seven seeds and all three metrics. Under this protocol the self-attentive models' advantage is consistent across seeds, but far smaller than the native-loss comparison indicates. The loss accounts for 84% of SASRec's 0.0575 HR@10 lead over BPR-trained HGN and 44% of its 0.0255 lead over BPR-trained U-GRU. The U-GRU implementation approximates cross-session state propagation (Section V-A), and its results are not evidence about the original architecture.

F. Catalogue Coverage and Intra-List Diversity

The loss changes coverage far more than any architectural choice. Among the fusion variants, the context gate covers more of the catalogue than either static alternative, and has higher ILD only in high-drift sessions. H4 concerns diversity, measured by ILD and coverage, both computed from each run's per-user top-10 lists over the 7,210 rankable items

TABLE XI
CATALOGUE COVERAGE@10 AND ILD@10 AT BATCH SIZE 512, WITH THE SEED SET OF EVERY ROW. *Top-10 Share* IS THE FRACTION OF ALL RECOMMENDATION SLOTS OCCUPIED BY THE TEN MOST FREQUENTLY RECOMMENDED ITEMS.

Model	Seeds	Coverage@10	Items	ILD@10	Top-10 share	HR@10
SASRec	7	0.228 ± 0.025	1,641	0.821 ± 0.010	0.165	0.0810
CAFREC-NP	3	0.234 ± 0.005	1,684	0.809 ± 0.004	0.152	0.0799
CAFREC	3	0.229 ± 0.010	1,654	0.813 ± 0.008	0.156	0.0791
Static scalar gate	3	0.201 ± 0.001	1,447	0.818 ± 0.003	0.163	0.0764
Concatenation	3	0.186 ± 0.015	1,340	0.816 ± 0.009	0.153	0.0722
HGN (CE)	7	0.422 ± 0.049	3,042	0.819 ± 0.002	0.151	0.0717
U-GRU (CE)	7	0.163 ± 0.010	1,177	0.850 ± 0.004	0.214	0.0668
HGN (BPR)	7	0.011 ± 0.001	77	0.809 ± 0.072	0.950	0.0235
U-GRU (BPR)	7	0.057 ± 0.010	408	0.878 ± 0.013	0.383	0.0554
MostPop	1	0.004	27	0.936	0.846	0.0432

Three-seed rows use $\{2020, 2021, 403092\}$ and seven-seed rows the headline set; MostPop is deterministic. Rows averaged over different seed sets are not comparable: CAFREC-NP's three-seed HR@10 of 0.0799 here is not comparable with SASRec's seven-seed 0.0810 (Table V).

TABLE XII
COVERAGE@10 AND ILD@10 IN THE HIGH- AND LOW-DRIFT STRATA ONLY (THREE SEEDS; EQUAL-SIZE SAMPLE OF 3,521 USERS PER STRATUM); VALUES OVER ALL USERS ARE IN TABLE XI. ILD USES FIRST-LEVEL CATEGORIES.

	Coverage@10		ILD@10	
Model	High	Low	High	Low
Context gate (CAFREC)	0.158	0.158	0.834	0.810
Static scalar gate	0.140	0.141	0.828	0.816
Concatenation	0.136	0.134	0.822	0.815

Coverage margin of the context gate in items, with two-sided paired permutation p-values (2,000 permutations within each seed). Against the static scalar gate: all $+206$, high $+131$, low $+120$; against concatenation: all $+314$, high $+164$, low $+177$; $p < 0.001$ in every case.

(Table XI). ILD was computed after the accuracy results were known. At the first category level, 0.03% or fewer of the recommended slots hold a video with no category. That is too few for the zero-vector convention to have a material effect.

Coverage of context-gated SASRec is similar to SASRec's: CAFREC-NP recommends 1,684 distinct items and SASRec 1,641. That 2.6% difference is small next to SASRec's seed-to-seed standard deviation of about 180 items. By contrast, under BPR the hierarchical baselines collapse onto a small popular subset. In these runs BPR-trained HGN devotes 95% of all slots to ten items (Table XI). Under cross-entropy the same HGN recommends 3,042 distinct items, nearly twice as many as any other model. ILD behaves differently. MostPop (27 items) and BPR-trained U-GRU (408 items) are two of the three narrowest catalogues. They have the highest ILD because lists assembled from a few popular videos still span many categories.

a) H4: H4 as stated requires the context gate to show higher ILD and coverage than the static fusion baselines, with the largest gains on high-drift sessions. Table XI reports both measures over all users. Table XII reports them by drift stratum, using one equal-size sample of 3,521 users per stratum shared by all three models. Coverage supports the first half. The context gate covers more of the catalogue than the static scalar gate and concatenation, by 206 and 314 items over all users. It does so in each of the three seeds and in both strata. A paired permutation test gives $p < 0.001$ for every margin, overall and within each stratum (note to Table XII). The test swaps each user's two top-10 lists between the models at random within each seed and recomputes coverage over all users (2,000 permutations). The margins therefore do not depend on which users were sampled: what limits the evidence is the seed. The standard applied above to the gap between CAFREC-NP and SASRec compares an effect with the seed-to-seed variation. By that standard, the margins of 206 and 314 items are of the order of SASRec's seed-to-seed standard deviation of about 180 items. Three seeds also cannot reach significance in a sign test (smallest two-sided $p = 0.25$). The margins are about the same size in the two drift strata, so coverage shows no drift ordering. ILD does not support the first half: over all users the context gate is 0.53% below the static gate and 0.36% below concatenation. Each difference has the same sign in only one or two of the three seeds.

Within high-drift sessions the ILD ordering reverses. The context gate exceeds the static gate by 0.82% ($+0.0068$, 95% CI $[+0.0046, +0.0090]$) and concatenation by 1.52% ($+0.0125$, $[+0.0096, +0.0152]$), in all three seeds in both cases. Within low-drift sessions it falls below both. The high-minus-low margin is positive for both comparisons ($+0.0118$ and $+0.0172$, both intervals excluding zero). Second-level categories and the KuaiRand `tag` encoding give the same pattern. At the second level, the high-drift gains are 0.57% against the static gate and 1.61% against concatenation. H4 is therefore **partly supported**. Coverage is higher as predicted in every seed, by margins of the order of the seed-to-seed variation. ILD is higher only in high-drift sessions, and the drift ordering predicted for the gains holds for ILD but not for coverage. The ILD differences are below 2%, rest on three seeds, and are about the size of ILD's seed-to-seed variation (Section VII-D).

G. Exploratory Analyses

The exploratory analyses suggest two things. At the default operating point, the gate's effect differs between session openers and later positions. In the lowest quartile, a constant vector supplies the frozen profile's advantage over the stand-in. None of them changes a conclusion, and no conclusion rests on them (Table III).

a) Session position: This analysis is exploratory: its partition was fixed after the aggregate result was known (Section IV-B). CAFREC-NP raises NDCG@10 relative to SASRec by 4.6% at session openers ($p_{\text{Holm}} = 0.0024$, seven of seven seeds) and lowers it by 3.0% within sessions. The pattern recurs with both models at batch size 2048 on the same data and partition. At openers CAFREC-NP also exceeds the shuffled-context control by 5.0% NDCG@10 in seven of seven seeds (exact Wilcoxon $p = 0.016$). Section VII-B reads this against the limits of that control. For CAFREC, the model H1 concerns, the opener difference (+2.9% NDCG@10) is not significant after Holm adjustment within the H1 family ($p_{\text{Holm}} = 0.47$). After equal-budget tuning, the opener advantage over SASRec falls to +2.1% ($p_{\text{Holm}} = 1.00$), against

$+0.5\%$ within sessions (95% CI $[-0.0009, +0.0016]$). The two positions then no longer differ. The effect is specific to the default configuration and is not carried forward as a property of context gating. Differences are paired against SASRec over the seven headline seeds.

b) Constant-profile control: In the lowest quartile, a fixed vector supplies all of a frozen profile's advantage over the trainable stand-in, about 11% (Section VI-C3). This control, specified after the H2 result was known, asks whether the margin reflects what the profile records about each user. The templated profile is already close to a constant vector, with a mean cosine of 0.991 between two users' profiles. The control replaces every user's generated profile with the mean of the corpus, which leaves one distinct row and no per-user information at all. This raises lowest-quartile NDCG@10 by 12.9% over the stand-in ($p_{\text{Holm}} = 2.2 \times 10^{-5}$, same sign in all three seeds). The constant profile is statistically indistinguishable from the templated profile ($+1.7\%$, $p = 0.28$), from the hashing profile, and from CAFREC-NP (-0.9%, $p = 0.45$). It exceeds the generated profile it was derived from by 13.4%. The advantage a frozen profile holds over the stand-in in this quartile is therefore not personalisation. A constant vector is also the data-minimal baseline for any profile, and studies of per-user profiles should report it.

c) History length and partial search: CAFREC-H adds the user's log training-history length to the gate input. Over seven seeds its descriptive gains over SASRec are 2.4% HR@10, 3.0% NDCG@10, and 3.5% MRR@10, but each is significant in only two or three seeds. It does not differ significantly from CAFREC-NP (HR@10 $+0.0013$, $p = 0.20$), and was selected from three candidates on the test split. A partial factorial search over six hyperparameters was run for SASRec and CAFREC-H and stopped on cost after 64 and 52 of 64 configurations. It selected dropout 0.2 for both. Over ten seeds, tuned CAFREC-H exceeded tuned SASRec by 2.9% NDCG@10 (across-seed Wilcoxon $p = 0.0076$). CAFREC-H was selected on the test split and CAFREC-NP was not searched, so this does not bear on the primary comparison.

d) Gate values: Over test users, g has mean 0.11 and standard deviation 0.27. Its mean is 0.12 at openers and 0.08 within sessions, and 0.08 and 0.12 in the high- and low-drift strata. Variation across sessions accounts for 3.8% of the variance of g and variation across dimensions for 66.0%. The remaining 30.2% is session-by-dimension interaction, where a context effect confined to some dimensions would appear, and it was not decomposed further. A factor shared by every dimension cannot alter a test ranking. Only changes in the relative weights of dimensions within a session can, and the decomposition above does not separate the two. Permuting one feature at test time changes NDCG@10 as follows: session length -0.44% $[-0.81\%, -0.07\%]$, dwell entropy $+0.08\%$ $[-0.16\%, +0.32\%]$, category drift -0.30% $[-0.51\%, -0.11\%]$, and policy flag -0.53% $[-1.19\%, +0.04\%]$. The inter-session gap changes it by -2.55% $[-3.67\%, -1.47\%]$, and the first-session feature by 0%, because that feature is 0 in every test case. Permutation is within session position, so that no opener receives a non-empty prefix. The four core features are constant across openers, so only within-session cases contribute for them. This measures the trained gate's reliance on each feature, and does not replace the pre-specified ranking, which retrains without each feature. These inference-only analyses use the saved CAFREC-NP checkpoints of the three ablation seeds, trained at batch size 512 on a CPU. They were defined after the aggregate result was known.

TABLE XIII
OUTCOMES OF THE FOUR HYPOTHESES. H1 TO H3 WERE PRE-SPECIFIED IN THE SUBMITTED PLAN; H4 WAS ADDED IN A LATER REVISION

Hyp.	Outcome	Evidence
H1	Not supported (vs SASRec)	Neither CAFREC nor CAFREC-NP differs significantly from SASRec after Holm correction (Table VI).
H2	Inconclusive	-0.4% lowest-quartile NDCG@10 against the stand-in and -12.6% against CAFREC-NP; the predicted advantage is not observed, and the training-target leak, expected to work against the generated profile, was not measured (Section VI-C3).
H3	Partly supported; mechanism not established	Beats concatenation in every stratum, and the static scalar gate only in aggregate and in the low-drift stratum; these margins reflect the alternatives' costs, not a gain from context (Table X).
H4	Partly supported	Higher coverage than both static alternatives in every seed, by margins of the order of the seed-to-seed variation; higher ILD on high-drift sessions only (Table XII).

VII. DISCUSSION

A. What the Study Establishes

Table XIII gives the outcome of each hypothesis.

This study holds the encoder, loss, batch size, and schedule fixed and uses the $g \to \mathbf{0}$ limit of the fusion as the baseline. Under that control, the aggregate benefit of exogenous session context on this dataset is too small to detect with 22,912 paired users and seven seeds.

The batch-size case of Section IV-F shows why matched settings come first. The apparent advantage it produced passed both the user-level and the seed-level tests this paper applies. Several seeds and significance tests cannot expose an artefact that shifts every seed the same way. Only matched settings did, and the result files of those runs did not record the setting.

Other settings moved results as much. The loss accounted for 84% of SASRec's lead over BPR-trained HGN (Section VI-E). Across seeds alone, the difference between the same two models ranged from -1.3% to $+2.9\%$ (Fig. 3). Single pairs of runs differed by 4% in test NDCG@10 between two GPU models and by 1.9% in HR@10 between CPU and GPU (Sections VI-B and IV-G). Gains of a few per cent for a gated recommender over its own base encoder are therefore within what one optimisation setting, seed, or device can produce. These observations are specific to this study, and none shows that any published gain is an artefact. They do, however, place optimisation settings alongside the tuning effort and baseline configuration already identified as sources of apparent progress in recommendation [22]–[24]. Multiple runs per configuration, reported seed variance,

and significance testing are established recommendations in machine learning and language processing [19]–[21]. This study does not propose them, but measures on one matched recommender comparison how far each setting can move a result. Their sizes may differ on other data, but the argument does not rest on their size. Where the seed or an optimisation setting moves a result by more than the effect under test, as here, one run per configuration cannot tell the two apart. Several seeds and paired tests, the other two practices this study relies on, each caught a different failure here. Across seeds the gate's effect changed sign (Fig. 3), which only several seeds expose. MRR@10 reached $p < 0.05$ in two of seven single seeds while the Holm-adjusted pooled test did not (Table VI), so a significant single run would not have replicated.

The same evidence standard applies to the systemic-risk assessment and mitigation of Articles 34 and 35 of the Digital Services Act (Section I), where mitigation may include testing and adapting recommender systems. It also applies to the independent audit of compliance with both under Article 37 [18], whose Delegated Regulation is discussed in Section II-D. Where a mitigation claim rests on an offline comparison between two configurations of a recommender, that comparison is the one this study makes, and the seed, batch-size, and device effects above apply to it. So does ILD's seed-to-seed standard deviation, which was about the size of the differences between models (Section VII-D). A claim that a change to a recommender mitigates a risk, resting on one training run of each configuration, is therefore weak evidence by the standard this study applies. That standard asks for matched settings recorded for every run, several seeds per configuration, and paired tests with intervals. These three items are a minimum, not a full protocol. The comparisons reported here are paired and multi-seed, but they meet the first item only in part. The runs predate per-run configuration logging, and their settings were reconstructed afterwards from versioned launch code into the manifest of Section IV-F. All of them also carry the timeline leak of Section IV-E, and those with a long-term profile carry the training-target leak of Section III-C. The items are proposed here for small effects measured by offline comparison, and are not requirements stated in the Act. The study makes no claim about the platform that released KuaiRand.

B. Interpretation of the Session-Position Result

In the exploratory analysis of Section VI-G, the gate's effect at the default operating point is not uniform. Without the profile it raised NDCG@10 at session openers and lowered it within sessions, and the two cancelled in aggregate. Three readings fit that pattern.

A session-start adjustment. The gate may apply a fixed rescaling at session starts, where the encoder's window holds only earlier sessions, that removes useful signal within a session. Retraining without the gap, which keeps the empty-prefix marker, changed nothing (Section VI-D).

An inter-session-gap signal. The gate could use the time since the last session to weigh how far the encoder's window reflects the current visit. Permuting the gap in the trained gate costs more than permuting any other feature (Section VI-G).

A context-dependent softmax temperature. The gate starts at $g \approx 0.5$, which halves every score (Section III-D), and ends slightly higher at openers than within sessions (Section VI-G). A rescaling shared by every dimension leaves a session's ranking unchanged, so this reading can act only through training, by changing the effective temperature of the loss.

The readings are not exclusive, and none was tested by an intervention. The shuffled-context control cannot separate them (Section VI-D), and the effect is confined to the default operating point (Section VI-G). The separating experiments are listed in Section VII-E.

C. Interpretation of the Profile Experiments

The evaluated long-term profile representations did not provide measurable gains beyond the short-term sequential encoder under the evaluated protocol. The template profile summarises the behavioural statistics the gate already observes per session, and neither the size nor the semantics of its text encoder changed its null result. Content-bearing profiles built from watch history beyond the twenty-item window, the one case in which a long-term branch can carry history the encoder does not see, gave no gain in the dense cohort. They carry the training-target leak in averaged form (Section III-C). Averaged caption embeddings are nearly one shared vector (Section VI-C2), which leaves the user-specific part of each caption profile a small residual. The category histogram separates users more and costs less, but it did not help either. Recency-weighted, mean-centred, and attention-pooled profiles remain untested.

The cohort analysis adds a caution for work that motivates long-term profiles by data sparsity: under leave-one-out evaluation with a twenty-item window, users with short histories were the *easier* group for every model. Whether sparse users need help, and what kind, depends on the evaluation protocol as much as on the model.

The protocol also bounds what H2 can be asked. A user in the lowest activity quartile has at most 12 organic clicks, so after the two holdouts the twenty-item window already covers that user's entire history. A profile can therefore add only what item identities do not carry, such as caption semantics, and never history the encoder misses. That route did not help: in this quartile a constant vector matches the templated and hashing profiles and exceeds the generated one (Section VI-G).

D. Limitations

The limitations are grouped by the validity they threaten [49].

a) Internal validity:

- *One leave-one-out split.* Each user's last click is held out for testing, so the point at which the data are split differs between users. Suppose user A's last click is on a Monday, and user B keeps watching until Friday. If a video becomes popular on Wednesday, B's clicks on it are training rows. When the model predicts A's Monday click, it has already learned that the video is popular,

which no live system could have known on Monday. On average 4.4% (median 1.2%) of the training rows post-date a user's test click in this way (Section IV-E). A most-popular ranker shows the leak at work: it scores HR@10 0.0432 with popularity counted over all training rows, and 0.0408 with only the rows before each test click. Both models in every paired comparison train on the same later rows, but such a leak can still change which model ranks higher [45]. The gate's null result and the profile comparisons are therefore established for this split only. None of them has been checked under a global temporal cutoff, a single date that separates training from test data for every user.

- *Training-target leak.* Each profile carries the leak at the grade given in Section III-C. No reported accuracy is inflated at evaluation, but the long-term branch is fitted under a signal it lacks at test time.

b) Construct validity:

- *No context-as-input comparison.* Admitting the same features through the encoder's input or attention, as TiSASRec and CARCA do [7], [8], was not tested, so the study cannot separate an uninformative context from an ineffective way of admitting it.
- *Limited profile representations.* The profiles are one templated summary, three unweighted content averages, and paragraphs from one generator, prompt design, and text encoder over three seeds (Section III-C). Other generators, prompts, fusions, and alignment with the collaborative representation were not tested.
- *Unexecuted pre-specified analyses.* Context features were not ranked by retraining, and the contribution of each apart from the policy flag is unknown.
- *Baseline fidelity.* HGN and U-GRU use default hyperparameters, and U-GRU approximates cross-session state propagation (Section V-A).

c) External validity:

- *One dataset tier.* All results come from KuaiRand-Pure, although the plan designated KuaiRand-1K as the primary tier. The sizes of the seed, batch-size, and loss effects may differ on other data, but the minimum standard of Section VII-A rests on their having occurred here, not on their size.
- *Operating point and training budget.* Both operating points stay within ten epochs with patience three, the second at different encoder depths and on different hardware (Section VI-B). At the default configuration every SASRec and CAFREC-NP run whose best epoch is known peaks at epoch 9 or 10 (Section IV-F), so undertraining at the default operating point is plausible. At the tuned configurations the best epoch ranges from 5 to 10. The best epochs of the headline A10G runs were not recorded. Default configurations have left a widely used sequential baseline underfitted [33], and no wider search, longer budget, or larger model was evaluated.

d) Statistical-conclusion validity:

- *A null is not an absence.* Any effect is bounded at about 2% of SASRec's HR@10 over all users, less tightly within sessions (Sections VI-A and VI-B).
- *Post hoc analyses.* The partitions and every analysis marked E in Table III were defined after the aggregate results were known, and ILD was computed after the accuracy results.
- *Fewer seeds for some analyses.* Ablations, controls, diversity measurements, and the CPU-trained content-bearing profiles use three seeds. ILD's seed-to-seed standard deviation (0.001 to 0.011) is about the size of the differences between models. The coverage margins are of the order of SASRec's seed-to-seed standard deviation, and three seeds are too few for a seed-level test (Section VI-F). Neither measure is therefore reported precisely enough here to support the assessment use noted in Section I.

E. Future Experiments

The design transfers to other platforms, because the context features use quantities that any interaction log carries, but whether the null result transfers is untested. The experiments needed to extend the claims are:

- Replicate the session-position analysis with the partition fixed in advance, at both operating points. Run it on a split with a global temporal cutoff and on a second tier such as KuaiRand-1K with a k-core floor. Extend the equal-budget search beyond ten epochs and the four factors varied here, and confirm CAFREC-H on a fresh split.
- Separate the three readings of Section VII-B. One test shuffles context only between sessions of the same position. A second compares a gate whose sole input is an opener indicator with one whose sole input is the inter-session gap. A third initialises the gate near $g = 0$ or trains SASRec at a softmax temperature of 2, with a residual concatenation baseline that starts from the same point.
- Admit the same context features through the encoder's input or attention, rank them by retraining, and try richer context such as the category composition of the session prefix.
- Run the content-bearing profiles on the seven headline seeds and the hardware of the other runs. Vary the generator, prompt, and fusion of the generated profile. Compare it with a mean of item embeddings built per prefix, an interaction-derived comparator that has no per-user parameters and does not carry the training-target leak. Test the constant-profile finding of Section VI-G beyond this dataset and architecture. Make profiles causal with respect to training examples, either by training only on each user's final training position with the profile built from the rows before that target, or by building a profile per prefix.
- Test a history window restricted to the current session. Article 38 of the Digital Services Act requires very large online platforms that use recommender systems to offer at least one option not based on profiling as defined in Article 4(4) of the GDPR [18], [43]. Whether a session-only window meets that definition is a legal question this study does not address. No evaluated model

is session-only, because CAFREC-NP's window spans earlier sessions and its gate reads the inter-session gap, and the accuracy of such a window is untested.

VIII. Conclusion

This study asked whether exogenous session context improves a matched SASRec when it enters only through a gate, and whether a long-term profile adds further value. On KuaiRand-Pure, under full-catalogue ranking, seven seeds, and paired per-user tests, context gating does not detectably improve SASRec, and H1 is not supported against it. At the default operating point, the interval for context-gated SASRec's advantage excludes gains above about 2% of SASRec's HR@10 over all test users. Two-thirds of those users' test clicks open a session, where the within-session features are still empty. The null holds after an equal-budget search, although the selected configurations differ in depth.

None of five long-term profiles gained over the model with no long-term branch. H2 is inconclusive. The generated profile showed no advantage in the lowest quartile and trailed the model with no profile, but the training-target leak, which was not measured, is expected to work against it. In an exploratory control, a constant vector matches the best profile in that quartile. H3 and H4 are partly supported, but the margins over the static alternatives measure their costs, and whether the gate uses session-specific context is not established.

The loss, the batch size, and the seed each moved results by more than the gate did. Claims of small gains measured by offline comparison, in publications and in systemic-risk assessments alike, therefore need at minimum matched settings recorded for every run, several seeds per configuration, and paired tests with intervals. Because every result here comes from one leave-one-out split, the first next step is a split with a global temporal cutoff. The others are a second dataset tier with the session-position partition fixed in advance, the seven-seed replication of the content-bearing profiles, and a longer training budget for both models.

Acknowledgment

The author thanks Michael Duignan, who supervised this project, for his weekly guidance throughout, and for raising the exposure-bias concern about KuaiSAR that led the study to KuaiRand.

Declaration of AI Use

AI tools were used in this project following King's College London's PAIR framework [50].

Problem formulation. The author set the research question, the hypotheses and the study design.

AI tool selection. Three tools were used, each for one task: Claude (Anthropic) for code, Google Gemini for finding related work, and Grammarly for proofreading.

Interaction. Claude helped debug the evaluation harness, develop its test suite, and the analysis and figure scripts from the author's specifications. Gemini was used to search for and summarise related work, and the author read every cited source.

Reflection. The author ran every experiment and reviewed all AI-written code against the test suite of Section V-B. He traced every reported value to its result file through the manifest of Section IV-F, and takes responsibility for all code, results and text. The language model that generates the profiles of Section III-C is part of the method and is described there.

Appendix

The source code, stored results, and supplementary material for this paper are publicly available on GitHub:

https://github.com/Ronan-Francis/CAFREC

The repository contains:

- the CAFREC model, the RecBole-based experiment harness, and the analysis scripts (`cafrec_harness/`);
- the data-preparation notebooks for KuaiRand-Pure (`Foundation/`) and instructions for rebuilding the datasets (`data/README.md`);
- the stored result file of every run, with a manifest that maps each reported value to its commit, launch script, batch size, seeds, and device (`cafrec_harness/results/`);
- the test suite (`cafrec_harness/tests/`) and a script, `audit_paper_claims.py`, that recomputes the reported numbers from the stored runs;
- a README with instructions for installing, running, and reproducing the experiments;
- this paper in PDF format (`G00403092.pdf`) and a screencast of the application running (`G00403092_Screencast.mp4`).

The KuaiRand-Pure dataset is not redistributed; it is available from its authors under CC BY-SA 4.0 [16].

References

[1] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2018, pp. 197–206.

[2] F. Sun *et al.*, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2019, pp. 1441–1450.

[3] M. Quadrana, A. Karatzoglou, B. Hidasi, and P. Cremonesi, "Personalizing session-based recommendations with hierarchical recurrent neural networks," in *Proc. 11th ACM Conf. Recommender Systems (RecSys)*, 2017, pp. 130–137.

[4] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *Neural Networks*, vol. 113, pp. 54–71, 2019.

[5] A. Lu, H. Yuan, T. Feng, and Y. Sun, "Rethinking the stability–plasticity trade-off in continual learning from an architectural perspective," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025.

[6] L. Jin, X. Li, S. Kim, and J. Kim, "Balancing long– and short-term user preferences via multilevel sequential patterns for review-aware recommendation," *Electronics*, vol. 15, no. 4, p. 753, 2026.

[7] J. Li, Y. Wang, and J. McAuley, "Time interval aware self-attention for sequential recommendation," in *Proc. 13th ACM Int. Conf. Web Search and Data Mining (WSDM)*, 2020, pp. 322–330.

[8] A. Rashed, S. Elsayed, and L. Schmidt-Thieme, "Context and attribute-aware sequential recommendation via cross-attention," in *Proc. 16th ACM Conf. Recommender Systems (RecSys)*, 2022, pp. 71–80.

[9] Z. Yu, J. Lian, A. Mahmoody, G. Liu, and X. Xie, "Adaptive user modeling with long and short-term preferences for personalized recommendation," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4213–4219.

[10] D. Li and Q. Gao, "Session recommendation model based on context-aware and gated graph neural networks," *Computational Intelligence and Neuroscience*, vol. 2021, p. 7266960, 2021.

[11] Z. Zhang, H. Zhang, Z. Zhang, and B. Wang, "Context-embedded hypergraph attention network and self-attention for session recommendation," *Scientific Reports*, vol. 14, p. 19413, 2024.

[12] Z. Zhang, J. Yu, and X. Li, "Context-aware session-based recommendation with graph neural networks," in *Proc. IEEE Int. Conf. Knowledge Graph (ICKG)*, 2023, pp. 35–44.

[13] J. Chang *et al.*, "PEPNet: Parameter and embedding personalized network for infusing with personalized prior information," in *Proc. 29th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 3795–3804.

[14] S. Dai *et al.*, "POSO: Personalized cold start modules for large-scale recommender systems," arXiv preprint arXiv:2108.04690, 2021.

[15] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[16] C. Gao *et al.*, "KuaiRand: An unbiased sequential recommendation dataset with randomly exposed videos," in *Proc. 31st ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2022, pp. 3953–3957.

[17] C. Ma, P. Kang, and X. Liu, "Hierarchical gating networks for sequential recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 825–833.

[18] European Parliament and Council of the European Union, "Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act)," *Off. J. Eur. Union*, vol. L 277, pp. 1–102, Oct. 2022. [Online]. Available: http://data.europa.eu/eli/reg/2022/2065/oj

[19] N. Reimers and I. Gurevych, "Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 338–348.

[20] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. 56th Annu. Meeting Association for Computational Linguistics (ACL)*, 2018, pp. 1383–1392.

[21] X. Bouthillier *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021.

[22] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *Proc. 13th ACM Conf. Recommender Systems (RecSys)*, 2019, pp. 101–109.

[23] S. Rendle, L. Zhang, and Y. Koren, "On the difficulty of evaluating baselines: A study on recommender systems," arXiv preprint arXiv:1905.01395, 2019.

[24] F. Shehzad and D. Jannach, "Everyone's a winner! On hyperparameter tuning of recommendation models," in *Proc. 17th ACM Conf. Recommender Systems (RecSys)*, 2023, pp. 652–657.

[25] A. Klenitskiy and A. Vasilev, "Turning dross into gold loss: Is BERT4Rec really better than SASRec?" in *Proc. 17th ACM Conf. Recommender Systems (RecSys)*, 2023, pp. 1120–1125.

[26] R. Francis, "A survey of hybrid user modeling methods for video recommendation: Balancing stability and plasticity," 2026, unpublished MSc literature review, Atlantic Technological University.

[27] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proc. 4th Int. Conf. Learning Representations (ICLR)*, 2016.

[28] B. Huang, J. Luo, W. Du, W. Pan, and Z. Ming, "Cascaded cross attention for review-based sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2023, pp. 170–179.

[29] M. Sabouri, M. Mansoury, K. Lin, and B. Mobasher, "Temporal user profiling with LLMs: Balancing short-term and long-term preferences for recommendations," arXiv preprint arXiv:2508.08454, 2025.

[30] D. Stamenkovic, A. Karatzoglou, I. Arapakis, X. Xin, and K. Katevas, "Choosing the best of both worlds: Diverse and novel recommendations through multi-objective reinforcement learning," in *Proc. 15th ACM Int. Conf. Web Search and Data Mining (WSDM)*, 2022, pp. 957–965.

[31] W. Krichene and S. Rendle, "On sampled metrics for item recommendation," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 1748–1757.

[32] B. Hidasi and Á. T. Czapp, "Widespread flaws in offline evaluation of recommender systems," in *Proc. 17th ACM Conf. Recommender Systems (RecSys)*, 2023, pp. 848–855.

[33] A. Petrov and C. Macdonald, "A systematic review and replicability study of BERT4Rec for sequential recommendation," in *Proc. 16th ACM Conf. Recommender Systems (RecSys)*, 2022, pp. 436–447.

[34] European Commission, "Commission delegated regulation (EU) 2024/436 of 20 October 2023 supplementing Regulation (EU) 2022/2065 of the European Parliament and of the Council, by laying down rules on the performance of audits for very large online platforms and very large online search engines," *Off. J. Eur. Union*, L series, 2024/436, Feb. 2024. [Online]. Available: http://data.europa.eu/eli/reg_del/2024/436/oj

[35] I. D. Raji *et al.*, "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," in *Proc. 2020 Conf. Fairness, Accountability, and Transparency (FAT*)*, 2020, pp. 33–44.

[36] D. Metaxa *et al.*, "Auditing algorithms: Understanding algorithmic systems from the outside in," *Foundations and Trends in Human–Computer Interaction*, vol. 14, no. 4, pp. 272–344, 2021.

[37] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2024, pp. 641–649.

[38] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Improving text embeddings with large language models," in *Proc. 62nd Annu. Meeting Association for Computational Linguistics (ACL)*, 2024, pp. 11 897–11 916.

[39] Qwen Team, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[40] Z. Sun *et al.*, "KuaiSAR: A unified search and recommendation dataset," in *Proc. 32nd ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2023, pp. 5407–5411.

[41] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[42] J. Morris, V. Kuleshov, V. Shmatikov, and A. Rush, "Text embeddings reveal (almost) as much as text," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12 448–12 460.

[43] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Off. J. Eur. Union*, vol. L 119, pp. 1–88, May 2016. [Online]. Available: http://data.europa.eu/eli/reg/2016/679/oj

[44] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertainty in Artificial Intelligence (UAI)*, 2009, pp. 452–461.

[45] Y. Ji, A. Sun, J. Zhang, and C. Li, "A critical study on data leakage in recommender system offline evaluation," *ACM Transactions on Information Systems*, vol. 41, no. 3, pp. 75:1–75:27, 2023.

[46] W. X. Zhao *et al.*, "RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms," in *Proc. 30th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2021, pp. 4653–4664.

[47] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.

[48] D. Lakens, "Sample size justification," *Collabra: Psychology*, vol. 8, no. 1, p. 33267, 2022.

[49] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.

[50] King's College London, "PAIR framework guidance," 2026, accessed: Oct. 12, 2026. [Online]. Available: https://www.kcl.ac.uk/about/strategy/learning-and-teaching/ai-guidance/pair-framework-guidance